
BASI DI DATI - Progetto n° 29

Corso di Laurea Informatica, A.A 2021/2022

Alessandro Giacconi, 152533

Gabriele Cusato, 152669

Federico Pierantoni, 154261

INDICE (cliccare sulle voci per andare alle sezioni)

Punto A, descrizione dell'ambito applicativo	2
Glossario	3
Punto B, schemi scheletro e schemi E/R	4
Strategia progettuale	4
1. Memorizzazione, separazione e classificazione delle carte	5
2. Descrizione del meccanismo di acquisto carte tramite buste	8
3. Parte relativa alla collezione di un utente e gestione dei mazzi	12
4. Mercato di scambio delle carte tramite la valuta di gioco	16
5. Utenti, classificazione e partite amichevoli e classificate	18
Schema finale	20
Punto C, progetto logico	22
1. Eliminazione delle gerarchie ISA	22
2. Selezione delle chiavi primarie ed eliminazione delle identificazioni esterne	26
3. Trasformazione degli attributi composti o multipli	29
4. Traduzione di entità e associazioni in schemi di relazioni	30
5. Verifica di normalizzazione	34
Punto D, operazioni previste dalla base di dati – Descrizione e relativo codice SQL	35
1. Query di selezione	35
2. Query di modifica	38
3. Query di eliminazione (per mostrare il DELETE CASCADE)	39
Punto E, codice SQL per la creazione delle tabelle e per l'inserimento dei dati	40
Creazione tabelle	40
Inserimento dati	54
Punto A extra, uso e implementazione dei trigger	61
1. Trigger sul limite delle carte in un mazzo (INSERT)	61
2. Trigger sul limite delle carte in un mazzo (UPDATE)	62
3. Aggiornamento del rank dopo un cambiamento dell'elo	63
4. Limite massimo di apparizioni di una carta in deck (INSERT)	64
5. Limite massimo di apparizioni di una carta in deck (UPDATE)	65
6. Limite massimo di acquisti di una busta speciale	66

Punto A, descrizione dell'ambito applicativo

Si assume di voler creare una base di dati per supportare una applicazione che permette di collezionare e giocare con le carte del famoso gioco di carte collezionabili "Yu-gi-oh". L'obiettivo è quello di fornire al pubblico un ecosistema che permetta di vivere appieno l'esperienza erogando il tutto in forma digitale, quindi non vi sono presenti alcuni riferimenti ad oggetti fisici.

L'ipotetica interfaccia virtuale del progetto è un'applicazione scaricabile in cui gli utenti possono registrarsi creando un account per collezionare e utilizzare le carte virtuali.

La base di dati di cui necessita l'applicazione deve quindi permettere di gestire diversi aspetti fondamentali:

- UTENTI

Gli utenti rappresentano le persone che hanno creato un account nell'applicazione, le quali devono essere in grado di possedere, comprare, vendere e utilizzare le carte. Le informazioni importanti di ogni singolo utente sono: il nome utente, la valuta che possiede (non gestita dal database ma dal lato applicativo) e il suo posizionamento nella classifica generale, stabilito sulla base delle sue abilità.

Quindi l'utente ha l'obiettivo di simulare una persona reale, inoltre si vuole implementare la possibilità di un utente di possedere una relazione di amicizia con altri utenti, creando quindi una relazione con essi che si discosta dalle sole partite.

- CARTE

Le carte rappresentano il fulcro dell'intera applicazione, come dello stesso mondo su cui si basa l'applicazione stessa. L'obiettivo è quello di permettere agli utenti di possederle e tramite l'utilizzo della valuta di gioco, citata precedentemente, di comprare o trovare all'interno di bustine virtuali delle nuove. Le carte possedute da un utente verranno salvate all'interno della collezione personale e potranno essere messe in vendita in cambio della valuta di gioco o utilizzate per comporre mazzi di carte poi utilizzabili all'interno della partite.

Risulta importante quindi anche implementare l'intera raccolta di tutte le tipologie di carte esistenti, per poi permetterne l'utilizzo all'interno dell'applicazione. Molte sono le informazioni richieste di memorizzare per le singole carte, tra cui il nome, l'immagine, edizione, rarità e descrizione dell'effetto.

- BUSTINE

Le bustine hanno lo scopo di creare una relazione diretta tra l'utente e le carte, permettendo quindi al giocatore di acquisirne delle nuove. Di conseguenza è rilevante differenziare le diverse tipologie di bustine, dal punto di vista di espansioni e dal punto di vista di tipologia, che si divide in normali e speciali. Delle bustine bisogna conoscere il codice seriale e il nome.

- PARTITE

Le partite potranno essere giocate da massimo 2 giocatori per volta, volendo fare distinzione fra le partite amichevoli, che non vanno ad intaccare il rank dell'utente, e quelle denominate classificate, che invece causeranno il cambiamento della valutazione nella classifica generale dell'utente in base all'esito dello scontro. Da questa ultima tipologia di partita risulta interessante dare la possibilità anche di partecipare a tornei, mantenendo sempre l'idea di variazione della valutazione dell'utente. Di una partita è fondamentale memorizzare la data in cui è avvenuta e l'id dell'utente vincitore.

Glossario

NOME	DESCRIZIONE	SINONIMI	LEGAME
Utente	nome, nickname, valuta	Utilizzatore Giocatore	Rank, Partita, Bustina, Collezione, Mazzo
Rank	nome, range_punteggio	Elo, grado	Utente
Partita	data, vincitore	Match	Utente, Torneo
Torneo	data, vincitore	Competizione	Partita
Bustina	nome, seriale		Utente, Carta
Carta	nome, descrizione, immagine, nome, rarità		Bustina, Collezione
Collezione	quantita_carte	Raccolta	Utente, Mazzo, Carta
Mazzo	nome, numero_carte	Deck	Utente, Collezione

Punto B, schemi scheletro e schemi E/R

L'attività complessiva attuata dal nostro applicativo è descritta tramite l'utilizzo di schemi scheletro seguite dalle rispettive parti di schema E/R commentate.

Il database si suddivide in 5 macro-categorie:

- Una parte che permette la corretta suddivisione, separazione e classificazione di ogni carta in modo da agevolare la ricerca ed avere una corretta memorizzazione di tutte le categorie di carte. Questo implementato tramite gerarchie e l'utilizzo di attributi enum.
- Una parte che descrive le carte, le buste in cui sono contenute e il meccanismo di acquisto e acquisizione delle carte nella propria collezione.
- Una parte utilizzata per la memorizzazione delle carte nella propria collezione e la possibilità di comporre mazzi dove ogni carta può apparire un massimo di tre volte.
- Una parte adibita al mercato ovvero un meccanismo di compravendita delle carte fra diversi utenti tramite l'utilizzo della valuta di gioco.
- Una parte utilizzata per la memorizzazione degli utenti, il loro grado e la loro possibilità di partecipare a partite amichevoli e classificate tramite dei tornei.

Strategia progettuale

La progettazione e la creazione di questo progetto sono state attuate tramite una metodologia modulare, ovvero dalla composizione delle singole parti in maniera separata si è giunti alla fine all'unione di tutte le parti tramite le opportune entità, cardinalità, associazioni e gerarchie in modo che lo schema E/R come tutto il progetto fossero il più completi possibile.

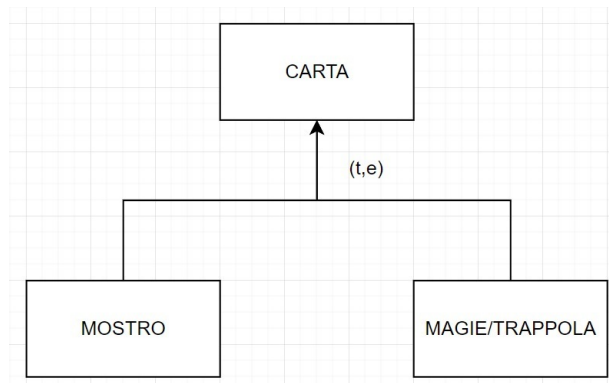
Si passa quindi alla descrizione e ai dettagli approfonditi su ogni singola parte del modello E/R.

1. Memorizzazione, separazione e classificazione delle carte

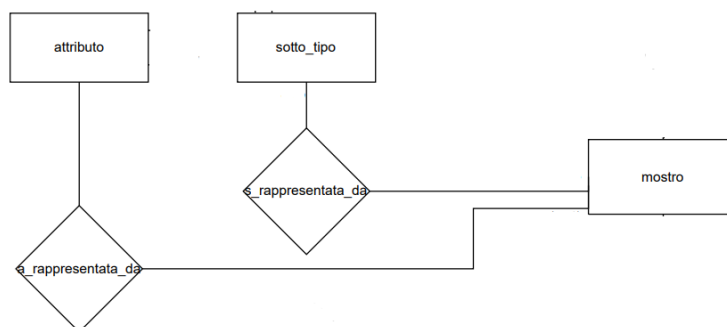
Di seguito è rappresentato lo schema a scheletro della sezione descritta che permette di capire le principali funzioni dello schema E/R presentato successivamente. L'entità carta rappresenta il concetto generale della carta da gioco ed è utilizzata per memorizzare tutte quelle presenti, al fine di dare la possibilità ad ogni singolo utente di cercarle ed utilizzarle.

1.1 Generalizzazione dell'entità carta

Dal momento che ogni carta può essere di due macro-tipologie: mostro o magia/trappole, si è deciso di generalizzare l'entità carta nelle due sottoentità tramite una generalizzazione di tipo totale, che non permette l'esistenza di tipologie non presenti nel database, ed esclusiva, che condiziona l'unicità della tipologia alla carta. Questa differenza è dovuta al fatto che la carta di tipologia mostro presenta molte più caratteristiche a differenza di quelle di tipo magia/trappola che devono essere memorizzate all'interno della base di dati.

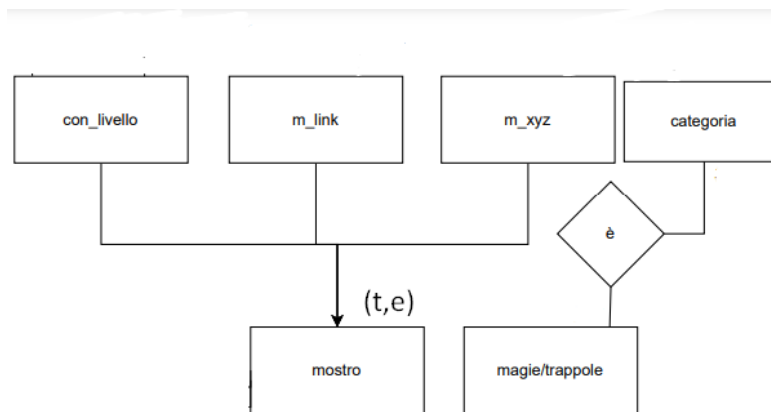


1.2 Caratteristiche della tipologia "mostro"



La tipologia mostro, oltre ad avere numerose sottocategorie trattate in seguito, possiede un sotto tipo e un attributo. Per mantenere l'integrità dei dati adeguata è stato scelto di implementarli tramite delle proprie tabelle che prevedono una caratterizzazione tramite un id e un nome. Ovviamente ogni mostro avrà un singolo sotto tipo e un singolo attributo.

1.3 sottocategorie di “mostri” e “magie/trappole”

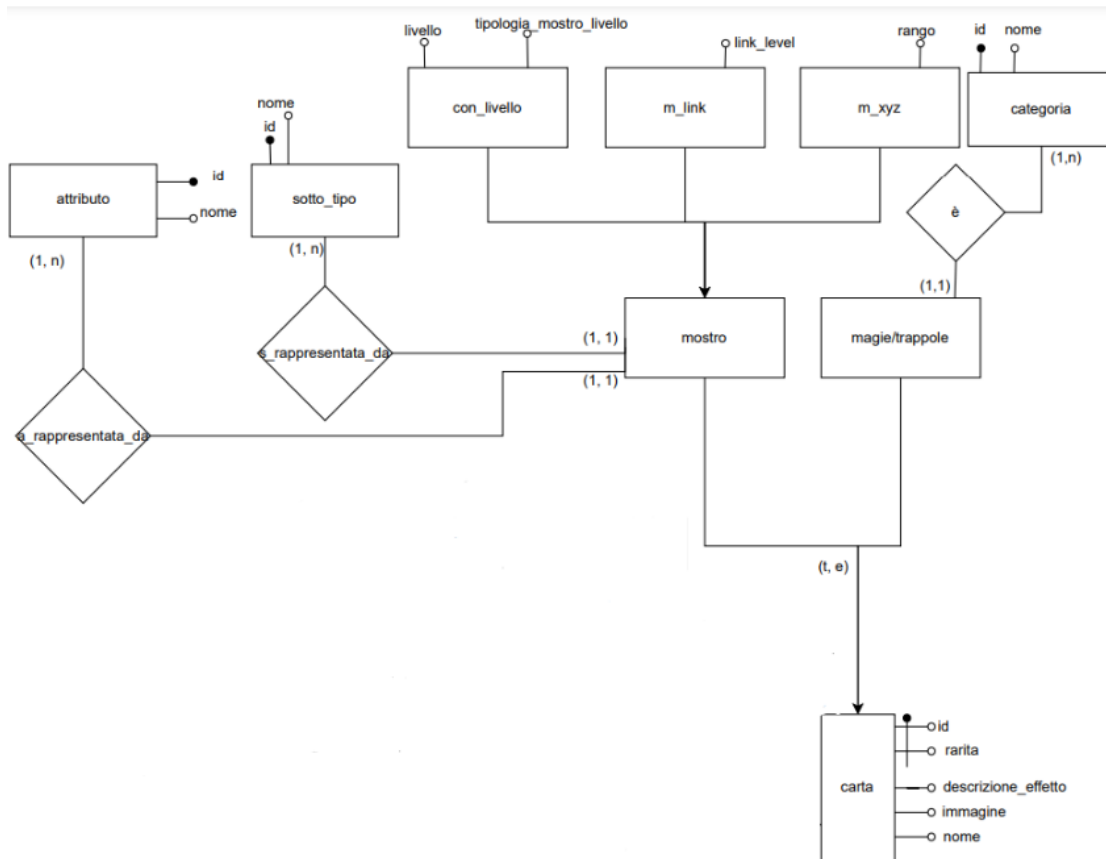


Questa è la struttura che caratterizza e classifica meglio i tipi di carte: una carta mostro può a sua volta essere di 3 tipi, conlivello, di tipo link o di tipo xyz. In quanto un mostro appartiene obbligatoriamente ad una di queste tre categorie si è optato per una gerarchia di tipo totale ed esclusiva. La differenza sostanziale è il tipo di livello che le varie categorie possiedono, infatti i mostri link hanno un proprio livello link diverso da quello normale, mentre i mostri xyz sono dotati, a differenza dei precedenti, di un rango. Le carte di tipo magia/trappola invece possiedono un numero di tipologie inferiore e per questo è stato deciso semplicemente di racchiuderle in una tabella e collegarla all'entità di partenza direttamente. Ovviamente una carta di tipo magia/trappola avrà un'unica categoria.

1.3.2 Proprietà e cardinalità

Ogni tipologia di mostro avrà quindi un opportuno attributo che assegna le caratteristiche elencate in precedenza. In più la sotto categoria “conlivello” ne possiederà uno aggiuntivo che successivamente verrà dichiarato come un enum, per identificare un ulteriore livello di sottocategorie che è importante caratterizzare per una semplice e rapida ricerca di una specifica carta. È stato scelto di non utilizzare un ulteriore livello di generalizzazione in quanto le differenze fra le sotto categorie aggiuntive sono minime, ma è giusto comunque differenziare e per questo è stato usato un attributo di selezione.

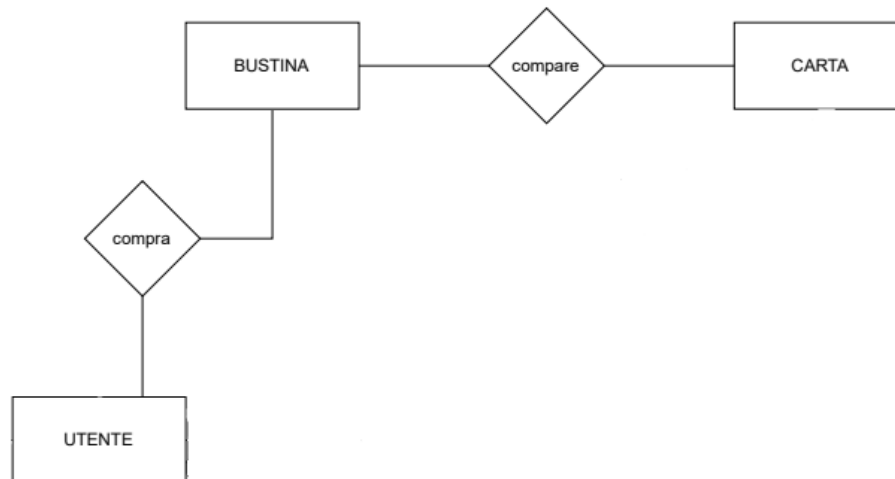
1.4 schema e considerazioni sulla parte relativa alle carte



Qui si nota l'implementazione dell'attributo "tipologia_mostrolivello" e i vari gradi di generalizzazione che ha subito l'entità carta, consentendogli di essere ricercata con una moltitudine di parametri per una ricerca tramite query successiva che potrà essere semplificata e più veloce. Da notare come l'entità "carta" possiede una chiave composta tra id e rarità, infatti una carta può apparire più volte in una collezione e può essere ricercata anche tramite la rarità. La parte più importante dove la rarità entrerà in gioco è la parte delle buste e delle espansioni di cui si parlerà in seguito.

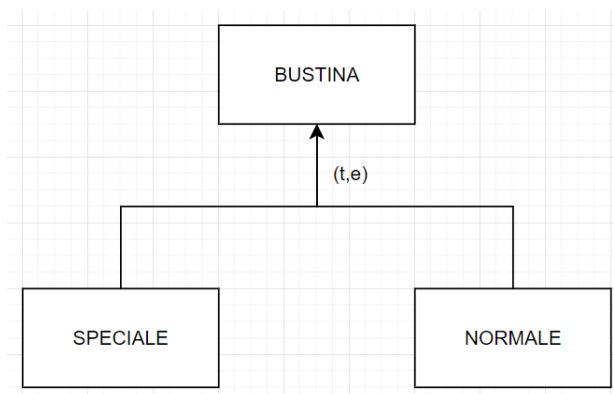
2. Descrizione del meccanismo di acquisto carte tramite buste

Di seguito è rappresentato lo schema a scheletro della sezione descritta che permette di capire le principali funzioni dello schema E/R presentato successivamente. L'idea iniziale era quella di permettere ad un utente di poter ottenere nuove carte tramite l'acquisto di buste che le contengono. Quindi l'entità bustina rappresenta il concetto di busta digitale che contiene le carte da gioco che un utente può trovare.



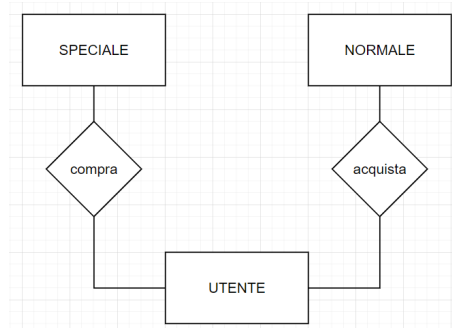
2.1 Generalizzazione dell'entità busta

Si è optato per una generalizzazione dell'entità bustina per permettere all'utente di poter acquistare 2 differenti tipologie di buste: la speciale, che conterrà un numero limitato di carte e sulla quale si limiterà il numero di acquisti per settimana; e la normale, che invece presenterà un numero maggiore di carte e senza alcun vincolo sulla possibilità di acquisto. Per la gerarchia si è deciso di crearla totale ed esclusiva. Le caratteristiche che accomunano i due tipi di buste sono il numero seriale, singolo per ogni bustina e il nome.

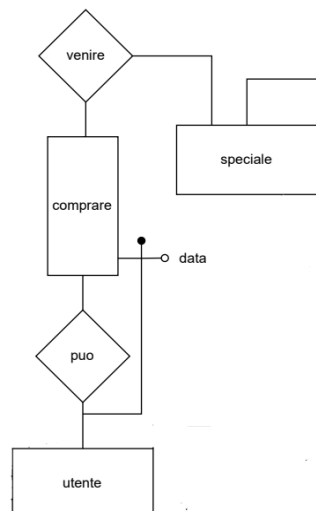


2.2 Sistema di acquisto di una busta da un utente

Dopo l'implementazione della gerarchia e dei vincoli spiegati precedentemente risulta obbligatoria la creazione di due associazioni differenti fra l'utente, la busta speciale e la busta normale. Entriamo più nello specifico parlando nelle seguenti sezioni della busta speciale e normale rappresentate nel seguente schema:



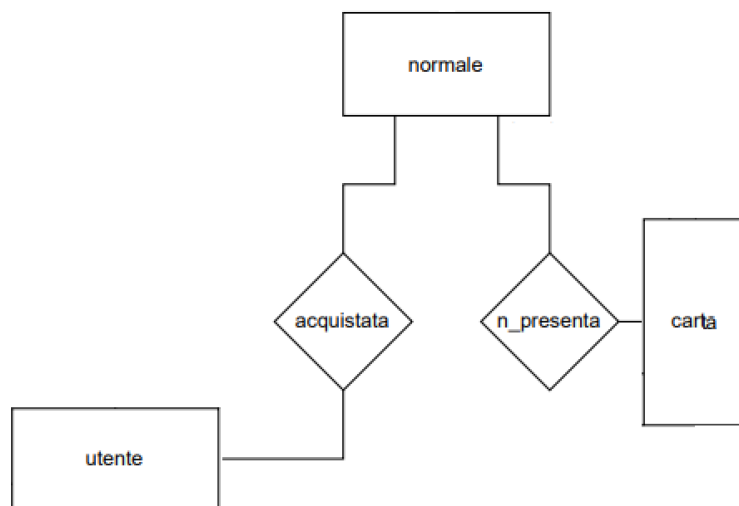
2.2.1 Schema e considerazioni sull'acquisto di una busta speciale



A causa del vincolo “una busta può essere acquistata da un utente al massimo 5 volte per data”, l'associazione “compra” deve essere reificata, ovvero si introduce la creazione di una nuova entità di nome “comprare” che avrà una chiave composta tra la chiave di utente e la data, espressa in settimane. Tramite questa reificazione siamo in grado di poter definire in modo corretto il vincolo, da completare solo tramite l'introduzione delle opportune cardinalità.

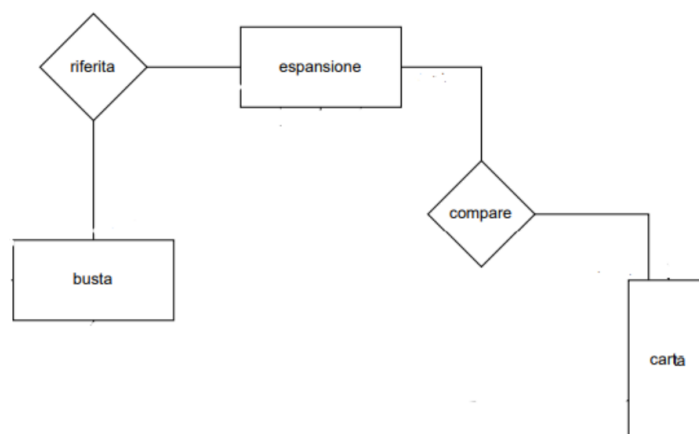
In quanto la busta speciale potrà essere acquistata al più cinque volte a settimana allora è stata utilizzata una cardinalità del tipo (1, 5) e in più l'entità “speciale” viene collegata con l'entità carta con una cardinalità (5, 5) dato che una di queste bustine contiene sempre 5 carte.

2.2.2 Caratterizzazione e acquisto di una busta normale



Al contrario della busta speciale, quella normale contiene nove carte e non ha limitazioni per quanto riguarda l'acquisto in ogni settimana, per questo l'associazione "acquistata" si trasformerà in una entità successivamente avendo agli estremi delle associazioni (0, n), in quanto una busta può essere acquistata da 0 ad n utenti mentre un singolo utente può acquistare quante volte vuole una busta. Dall'altra parte però si avrà una cardinalità del tipo (9, 9) in quanto una busta conterrà sempre nove carte. In più è da notare come dalla parte di carta vi è una cardinalità del tipo (1, n) sia per le carte normali ma anche per le speciali. Questo perché una carta può ritrovarsi in più buste e quindi in più espansioni, concetto che verrà trattato nel prossimo punto.

2.2.3 Concetto di espansione e collegamento con le buste



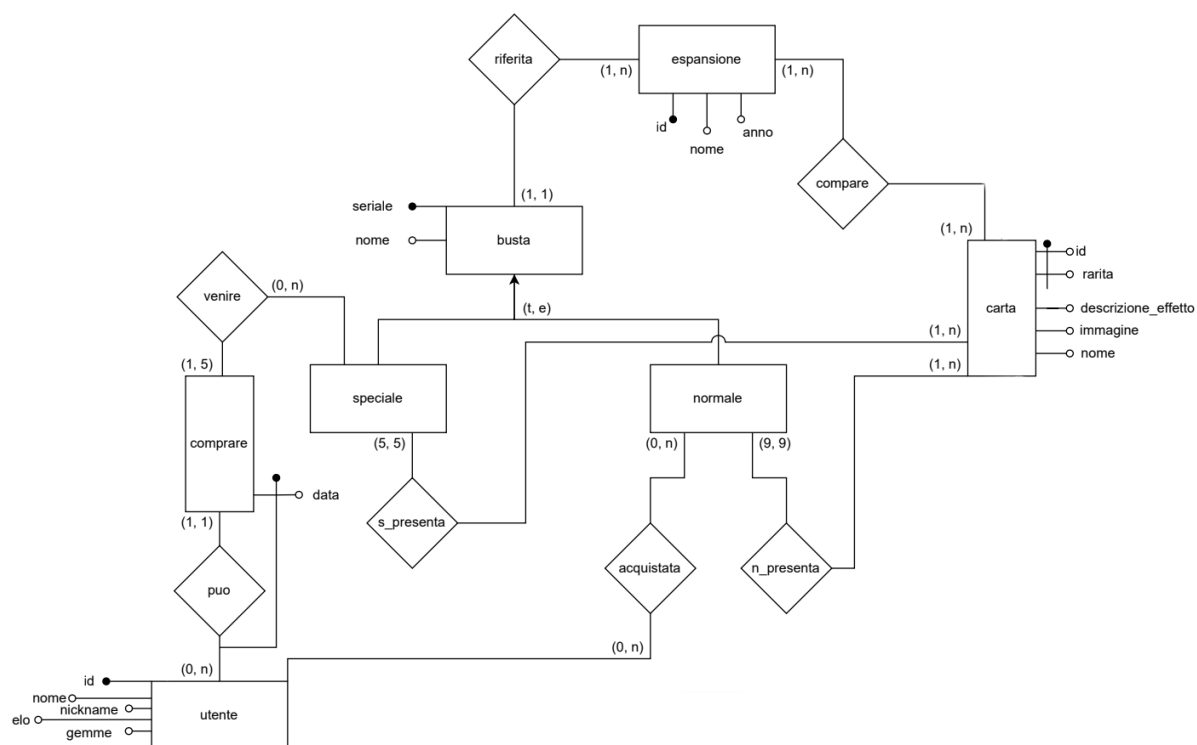
Un'espansione rappresenta un insieme di carte che verranno poi vendute nelle buste. Le carte all'interno rispecchiano certi archetipi seguendo un determinato criterio, per esempio l'espansione "draghi del chaos" (nome ipotetico) avrà al suo interno un certo numero di carte che formano uno o più archetipi. Questi, presi singolarmente o formando degli incroci, permettono la creazione di mazzi sempre più forti in base alle sinergie presenti tra le varie carte. Starà all'utente scoprire le migliori tattiche per creare un mazzo degno di nota.

2.2.4 attributi e cardinalità legati ad un'espansione

la relazione “compare” diventerà una tabella in quanto la cardinalità è del tipo (n, m) dato che una carta deve comparire in almeno un'espansione ma può comparire anche in più di una di esse. Ovviamente un'espansione conterrà da una ad n carte. Una busta si riferisce ad una sola espansione mentre un'espansione è ovviamente rappresentata da una ad n buste (utilizzando l'esempio precedente, l'espansione “draghi del chaos” renderà disponibili all'utenza le carte che ne fanno parte tramite n bustine di quell'espansione). Parlando di attributi l'entità espansione è caratterizzata da un nome, un anno di rilascio e un id come chiave primaria per semplicità e velocità nella manutenzione del database.

2.2.5 Schema e considerazioni sulla meccanica di acquisto di buste

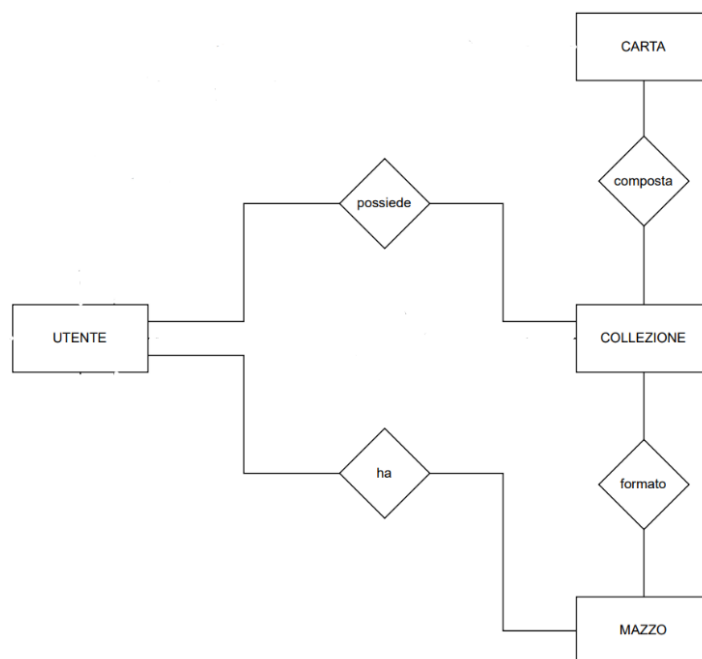
Lo schema che segue rappresenta l'implementazione di tutte le considerazioni effettuate precedentemente. È importante notare che le bustine potranno essere acquistate tramite la valuta di gioco, ossia le gemme dell'utente. Il costo delle buste varierà in base alla loro categoria, speciale o normale, e anche in base all'espansione. Tale aspetto verrà gestito nel lato applicativo.



L'attributo nome dall'entità busta è stato successivamente eliminato per una questione di ridondanza con l'entità espansione.

3. Parte relativa alla collezione di un utente e gestione dei mazzi

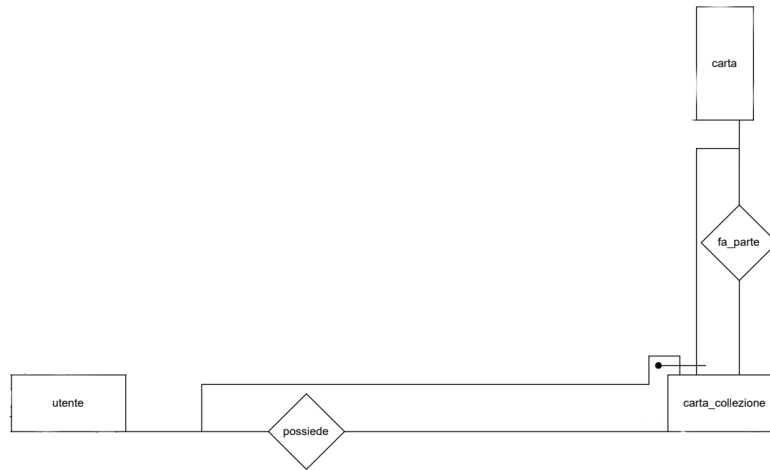
Di seguito è rappresentato lo schema a scheletro della sezione descritta, permette di capire le principali funzioni dello schema E/R presentato successivamente. L'obiettivo è quello di permettere ad un utente di possedere una collezione propria di carte, dalla quale ha la possibilità di generare dei mazzi poi utilizzabili all'interno delle partite.



3.1 Collezione di un utente

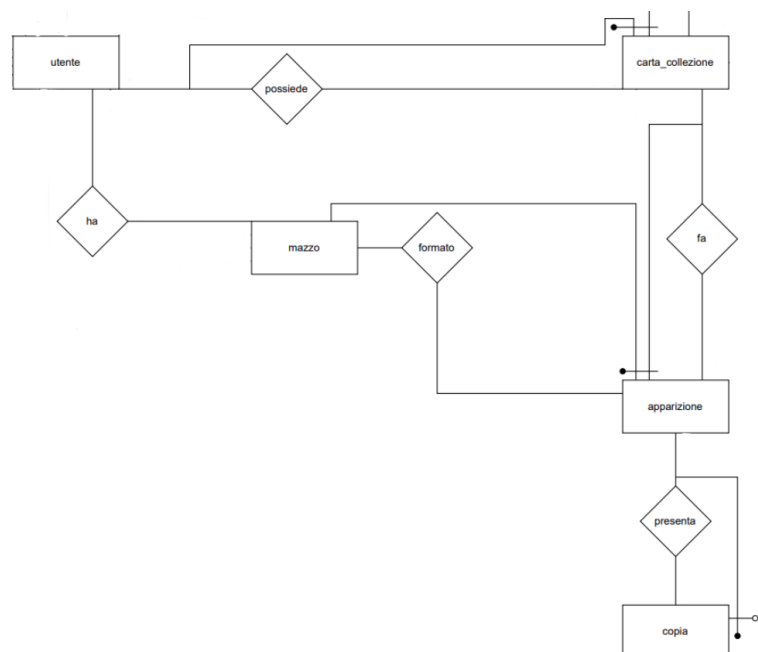
Introduciamo come è stata prodotta l'idea della collezione di un utente. Da essa dipenderà poi il punto successivo della gestione dei mazzi. Al fine di rappresentare la collezione digitale di un utente, una buona strategia è quella di creare una entità che si pone fra l'utente, il possessore della collezione, e l'entità carta, il concetto di carta generica, che avrà come chiave primaria la composizione delle chiavi delle due entità appena citate (sarà quindi la chiave composta fra rarità, id_carta e l'id_utente). Questo è stato fatto per poter dire "quella generica carta è posseduta da quel determinato utente", ma soprattutto per gestire importanti vincoli che rendono il database solido e corretto.

3.1.1 Cardinalità, proprietà e dettagli sulla collezione



Per come è stato gestito il vincolo si è posta una cardinalità (0, n) dalla parte dell'utente e dell'entità carta, in modo da poter dire che un utente può possedere più carte nella sua collezione (mantenendo valido il caso di non possedere carte alla creazione dell'account). Stesso concetto vale per il fatto che zero o più carte possono far parte della collezione di un utente. Ogni carta che fa parte della collezione è caratterizzata da una quantità che descrive il numero di carte uguali (inclusa la rarità essendo chiave) che l'utente possiede.

3.2 Gestione delle carte che formano un mazzo



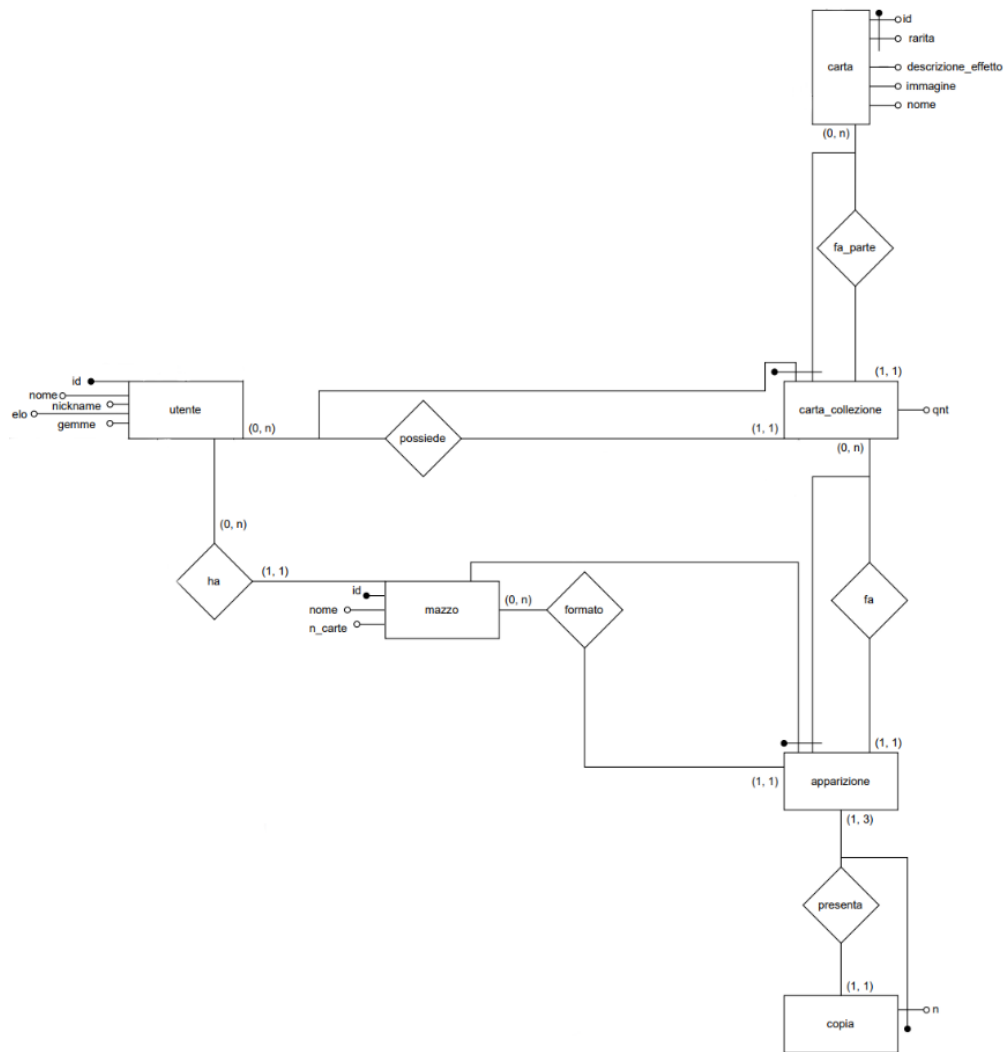
Per fare chiarezza nell'immagine precedente, è stato usato un meccanismo per limitare il numero di apparizioni di una certa carta in un mazzo. L'entità apparizione serve per capire se una determinata carta fa parte di un particolare deck, mentre l'entità "copia" serve per determinare il numero di apparizioni che una stessa carta fa in un mazzo. Quindi l'utente può possedere da zero ad n mazzi, il mazzo verrà formato con massimo quaranta apparizioni (dove in questo caso saranno di una singola copia ciascuna), ogni apparizione potrà essere formata da una a tre copie della stessa carta in uno stesso deck.

3.3.2 Proprietà, chiavi e cardinalità

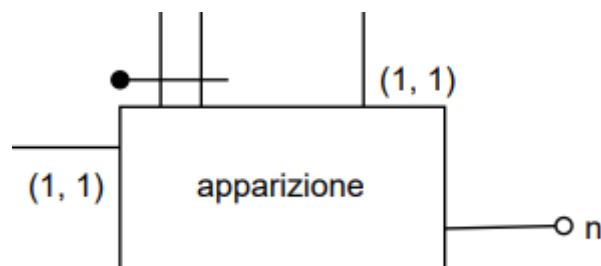
L'entità mazzo sarà composta quindi da un nome, da un numero di carte fissato che va da quaranta a sessanta e da un id come chiave primaria. Esso potrà avere da zero ad n apparizioni di carte (il numero massimo di apparizioni sarà gestito con dei trigger). L'entità apparizione rappresenta un'ulteriore reificazione oltre a carta_collezione che permette di gestire il fattore delle copie di carte possibili in un deck. La cardinalità tra apparizione e copia sarà quindi del tipo (1, 3) utilizzando come chiave di copia la chiave di apparizione insieme ad un numero n incrementale. Ovviamente contenendo apparizione una carta generica, la cardinalità tra apparizione e carta_collezione sarà una (0, n) dalla parte di carta_collezione e (1, 1) dalla parte di apparizione.

3.4 Schema finale e considerazioni su vincoli non rappresentabili con esso

Nel seguente schema sono stati implementati tutti gli attributi, gerarchie e cardinalità scritte in precedenza, insieme alle reificazioni discusse. In questa sezione sarebbe stato inutile e complesso gestire certi tipi di vincoli, in quanto facilmente realizzabili livello software tramite trigger o definizione dei dati all'interno del database.



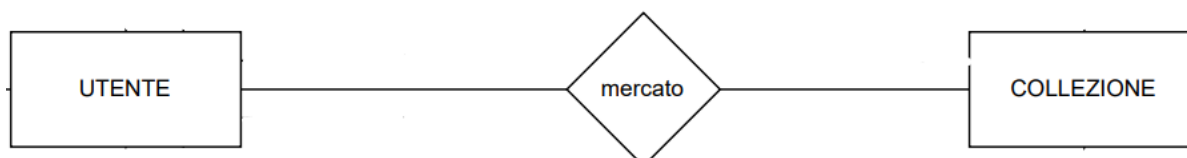
A seguito dell'implementazione del database è emersa l'utilità di modificare il meccanismo di enumerazione delle copie di una singola carta in un deck eliminando l'entità copia e spostando l'attributo "n" all'interno di "apparizione". Esso verrà controllato e opportunamente modificato tramite trigger. Il modello E/R che rappresenta questa modifica è il seguente:



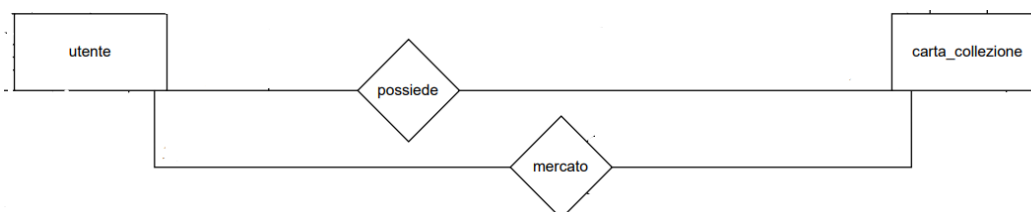
Alcuni trigger fondamentali realizzabili sono quelli sul limite di apparizioni e copie di carte presenti in un mazzo, tenendo conto della rarità (la somma degli stessi tipi di carta, anche se di rarità diverse, apparsi in uno stesso mazzo dev'essere al massimo tre) ; una carta non può apparire più volte rispetto al numero di carte possedute in carta_collezione ; un mazzo può avere dalle 40 alle 60 carte, ecc...

4. Mercato di scambio delle carte tramite la valuta di gioco

Di seguito è rappresentato lo schema a scheletro della sezione descritta, permette di capire le principali funzioni dello schema E/R presentato successivamente. L'entità collezione rappresenta un insieme di tutte le carte possedute da un utente dove ognuna è posseduta in un certo numero di copie e rarità, ma in questa sezione ci si concentrerà sulla funzione del mercato che, da questa semplice rappresentazione con schema a scheletro, va ad ampliarsi tramite un meccanismo complesso di compravendita tramite una valuta.



4.1 specializzazione della funzione di mercato



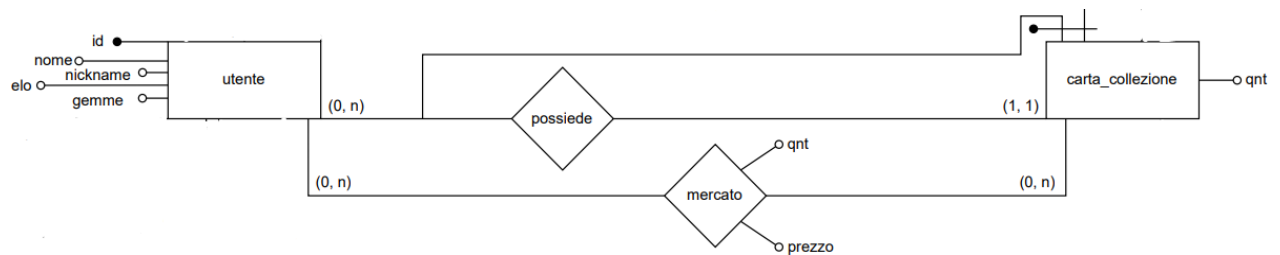
Come prima cosa è da notare come la semplice connessione fra utente e collezione si è sdoppiata in due associazioni diverse: un utente può possedere una collezione in quanto appena creato un account essa sarà vuota e quindi inesistente. Essa verrà creata e affidata al rispettivo utente dopo che sarà aperta la prima bustina o sarà acquisita la prima carta.

4.1.2 caratterizzazione di mercato e cardinalità

La relazione mercato fra le due entità avrà da entrambe le parti una cardinalità "(0, n)" il che porterà mercato a diventare un'entità, a cui si sono aggiunti due attributi fondamentali come la quantità di carte messe sul mercato e il prezzo condiviso da tutto il gruppo di carte uguali (se la quantità è maggiore di uno). L'utente dovrà essere capace tramite l'applicativo di ricercare le proprie carte messe sul mercato per semplicemente visionarle o eliminarle da esso, ma la cosa più importante sarà poter vedere le carte di tutti gli altri utenti messe sul mercato secondo opportuni criteri di ricerca, in modo che l'utente stesso possa acquistare poi le carte avendo una quantità di gemme superiore alla richiesta.

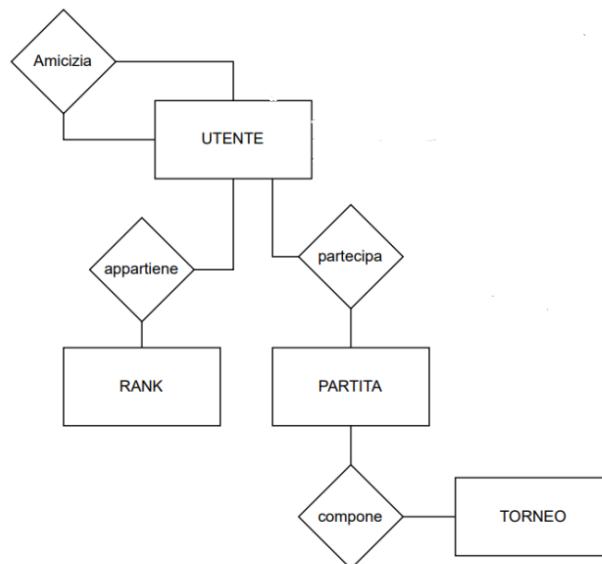
4.2 schema E/R e specifiche sull'entità "carta_collezione"

Nel seguente schema sono stati implementati attributi e cardinalità descritte in precedenza, ma è importante parlare dell'entità carta_collezione in quanto essa è una reificazione e per questo possiede una chiave composta dalla chiave di utente e la chiave dell'entità "carta" non visibile in questo schema. "carta_collezione" conterrà quindi le carte possedute dall'utente in questione secondo una certa quantità definita dall'attributo "qnt" e ogni gruppo di carte dipenderà sia dalla carta stessa ma anche dalla rarità, in quanto id e rarità sono chiave composta dell'entità "carta". L'entità "carta_collezione" è stata approfondita nella sezione 3.

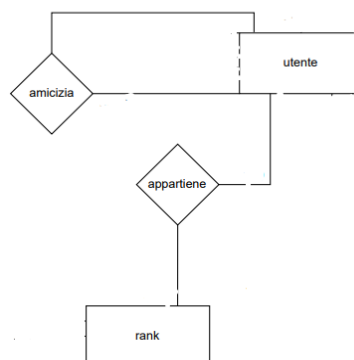


5. Utenti, classificazione e partite amichevoli e classificate

Di seguito è rappresentato lo schema a scheletro della sezione descritta, permette di capire le principali funzioni dello schema E/R presentato successivamente. Gli utenti hanno la possibilità di stringere amicizie con altri utenti, un rank che rappresenta il grado di abilità dell'utente e una composizione delle partite disputate che possono anche essere di un determinato torneo.



5.1 rappresentazione semplice di utenti, amicizie e classificazione tramite rank

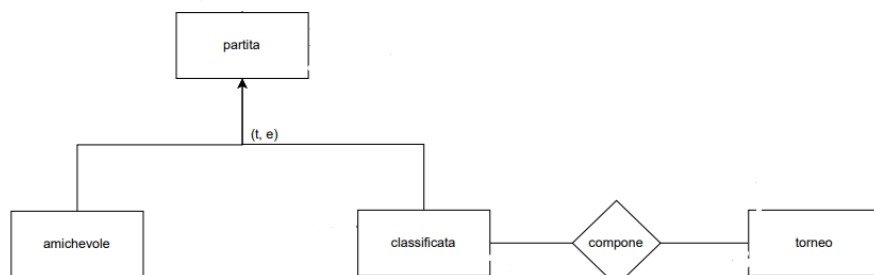


“Utente” è un’entità che permette la memorizzazione di ogni utente con tutte le sue caratteristiche, ma per solidità e correttezza dei dati, la caratteristica di grado, chiamata in questo schema scheletro: rank, è stata implementata tramite un’entità che avrà poi degli attributi per identificare un range sui punti (detti elo) per il quale l’utente sarà di quel rispettivo rank. Ogni utente ha la possibilità di stringere delle amicizie che sono state implementate tramite l’utilizzo di un’auto-associazione che descrive nel modo più semplice il meccanismo voluto.

5.1.2 Proprietà e cardinalità

Un utente quindi può avere un numero non specificato di amicizie mentre può avere ovviamente un solo rank caratterizzato dalla quantità di elo dell'utente. Un utente è identificato da un nickname che sarà univoco fra tutti gli utenti presenti nel database, inoltre ogni giocatore avrà una certa quantità di gemme, inizialmente nulla, con cui potrà acquistare bustine virtuali o singole carte all'interno del mercato. È importante notare che nonostante il nickname di un utente sia univoco esso non è stato utilizzato come chiave primaria, in quanto un id numerico progressivo è più adatto ad una ricerca e classificazione degli utenti sul lato gestionale.

5.2 sistema delle partite amichevoli e classificate con relativo elo guadagnato o perduto



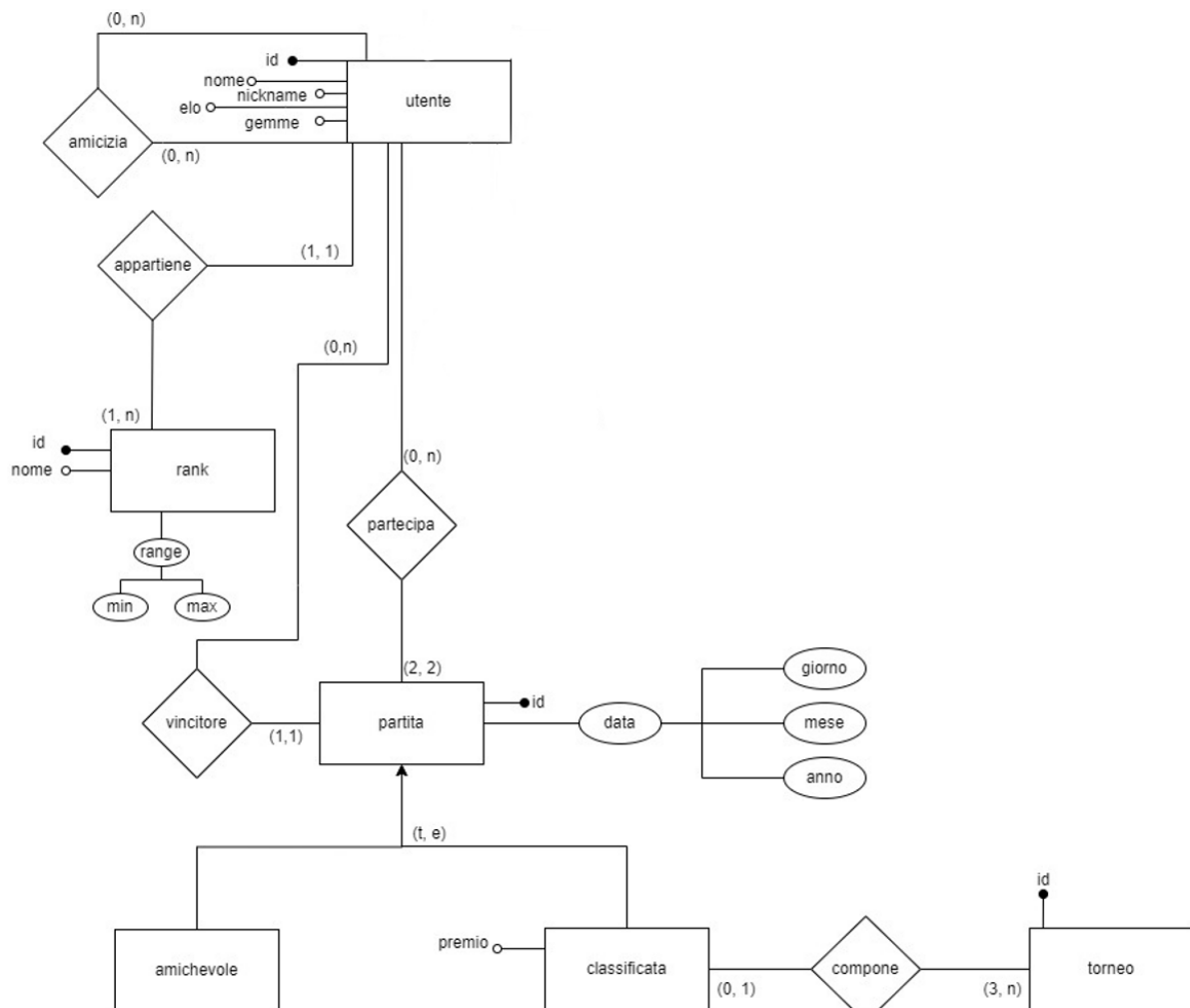
Dato che sia le partite amichevoli che quelle classificate avranno stessa chiave primaria e attributi, per rappresentare la differenza è stata utilizzata una generalizzazione del tipo totale ed esclusiva in modo che una partita dev'essere o di tipo amichevole oppure di tipo classificata e non può appartenere a nessun altra categoria. Tenere traccia delle partite disputate da un utente è utile nel caso in cui si voglia riguardare il match oppure controllare il deck dell'avversario con i suoi rispettivi dati.

5.2.2 Proprietà e cardinalità

L'entità "partita" è caratterizzata da una relazione "vincitore" che serve per identificare quale dei due utenti è risultato vittorioso, stabilendo una cardinalità del tipo (0,n) dalla parte di "utente" e (1,1) dalla parte di "partita". L'entità "partita" è collegata ad "utente" tramite una cardinalità del tipo (2, 2) per specificare che una partita è svolta da solamente 2 giocatori, in più ognuna di esse verrà caratterizzata da un attributo "data" che semplificherà la ricerca e la classificazione di ogni determinata partita. Nel caso della partita classificata si tiene traccia del premio, ovvero i punti elo, guadagnati o perduti dall'utente dopo aver portato a termine la partita. La partita classificata può essere stata giocata tramite un torneo oppure tramite una partita singola contro un altro giocatore. Il torneo dovrà quindi avere da 3 ad N partite, e quindi almeno 4 partecipanti. Per tenere traccia dei tornei si è sfruttata la chiave primaria: id, perché come nelle altre tabelle facilita e rende più rapida la gestione e ricerca dei dati.

5.3 schema e considerazioni sulla parte relativa agli utenti

Di seguito sarà rappresentata la porzione di schema ER che implementa cardinalità ed attributi descritti nei punti 5.1 e 5.2. È importante notare che l'attributo nickname sarà reso univoco nella scrittura del codice SQL di creazione del database.

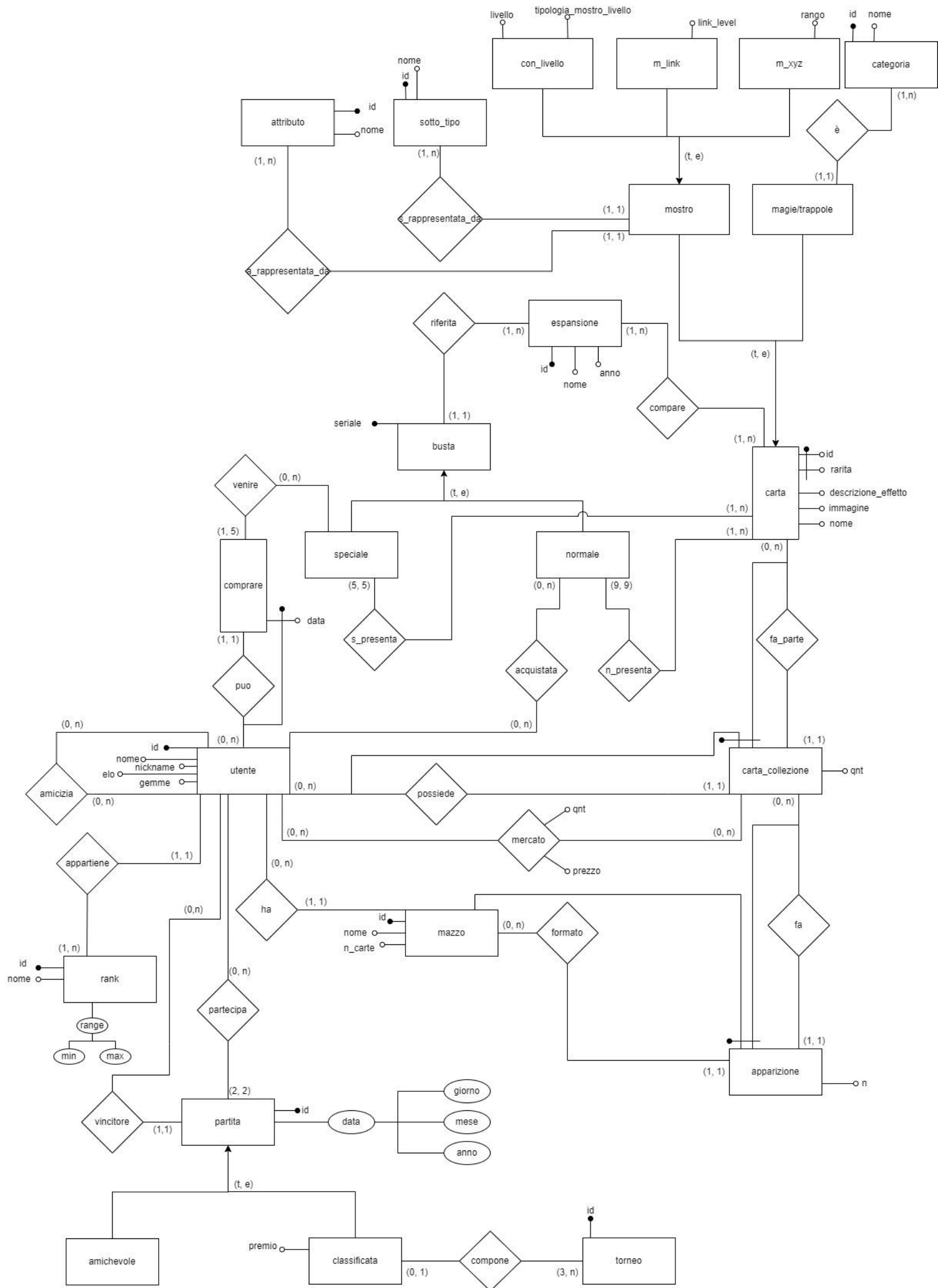


Schema finale

Di seguito sarà rappresentato lo schema E/R finale comprensivo di tutte le parti descritte nei 5 punti fondamentali della sezione B. Ogni parte che compone lo schema è stata realizzata inizialmente ad alto livello con schema a scheletro, e piano piano si sono andati a raffinare i particolari. Dopo aver costruito tutte le singole parti, esse sono state unite resolvendo i problemi di incompatibilità e ridondanza. L'immagine seguente è molto grande, perciò per una più comoda visione si allega l'immagine nella cartella della consegna.

Nota:

Importante notare che l'entità copia è stata omessa per una questione di ottimizzazione emersa durante l'implementazione del modello.



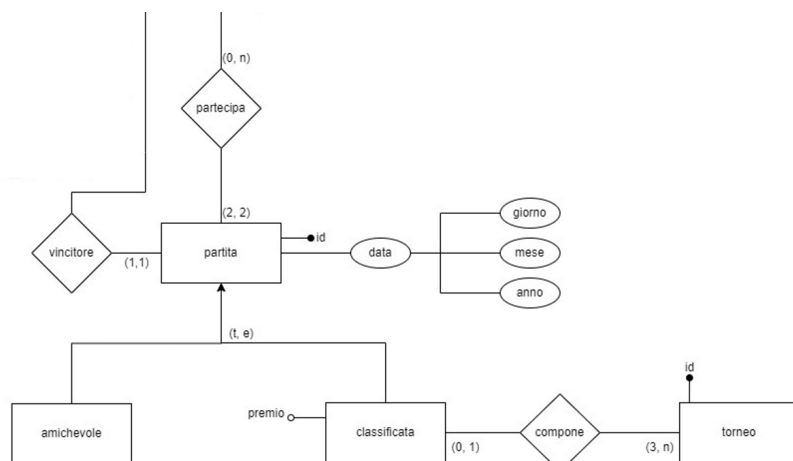
Punto C, progetto logico

La progettazione logica del database verrà descritta in modo sequenziale per poter presentare una trasformazione logica a partire dallo schema ER per arrivare al modello logico finale. I passaggi che verranno effettuati sono i seguenti:

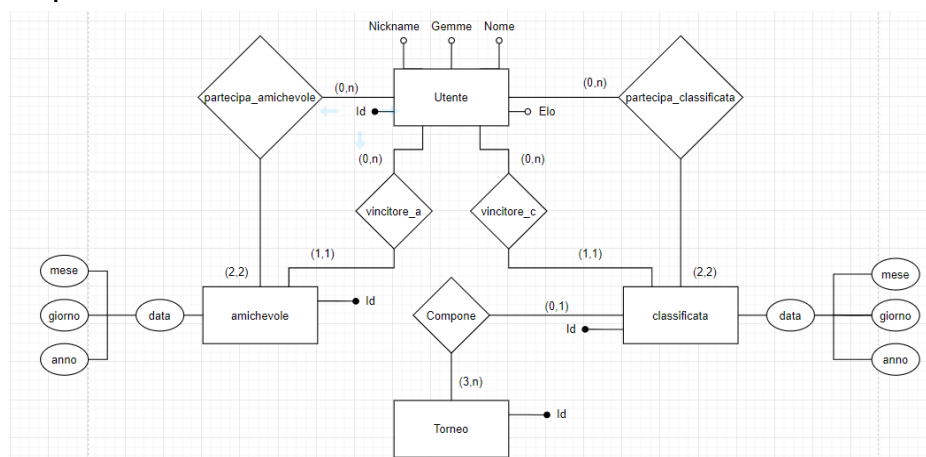
1. Eliminazione delle gerarchie ISA.
2. Selezione delle chiavi primarie, eliminazione delle identificazioni esterne.
3. Trasformazione degli attributi composti o multipli.
4. Traduzione di entità e associazioni in schemi di relazioni.
5. Verifica di normalizzazione.

1. Eliminazione delle gerarchie ISA

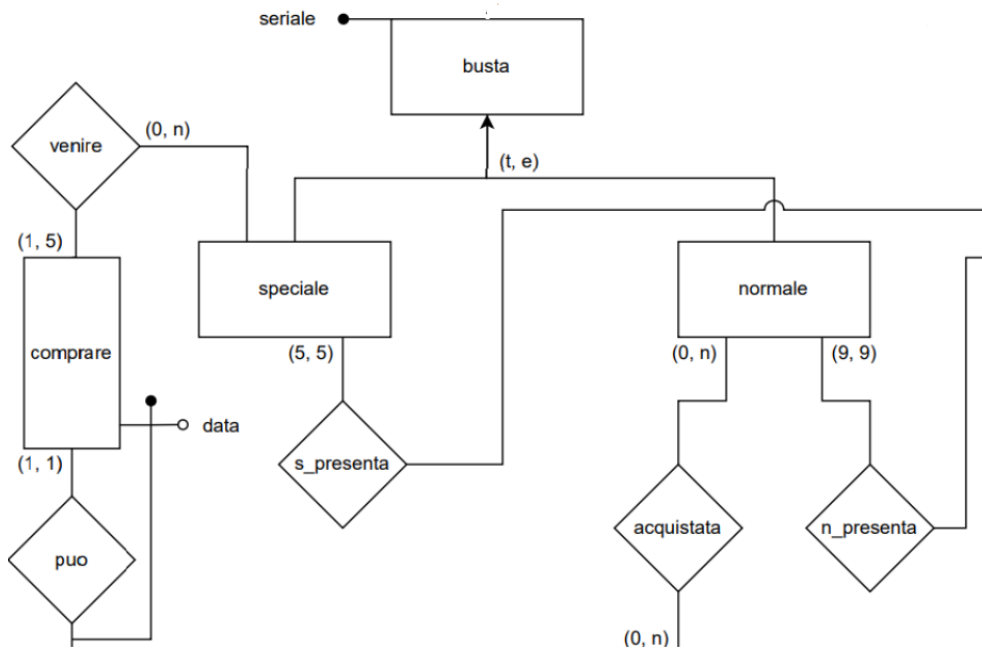
1.1 Gestione gerarchia partita



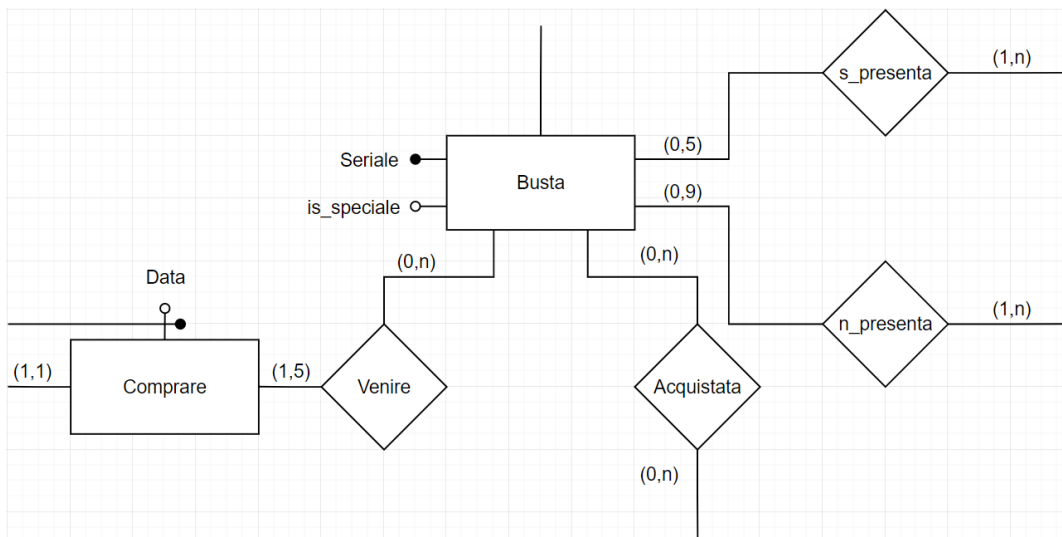
Sull'entità partita si è scelto di eliminare la generalizzazione tramite un collasso verso il basso. La scelta è stata fatta principalmente perché nella maggior parte delle interrogazioni o comunque azioni sul database non ci si sarebbe fermati all'esaminazione sull'entità "partita" ma si sarebbe andati ad agire sulle entità figlie. In più gli attributi di partita non sono troppo numerosi e per questo tutte le associazioni legate a partita sono state raddoppiate per collegarle a entrambe le entità figlie. La gerarchia di tipo totale ed esclusiva non ha comportato ulteriori modifiche.



1.2 Gestione gerarchia busta

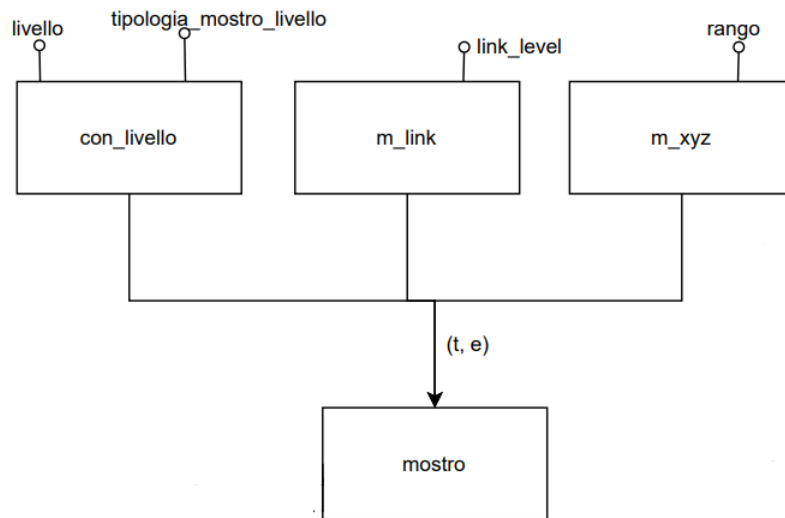


Riguardo l'entità busta si è scelto di adottare un'eliminazione della gerarchia secondo il metodo del collasso verso l'alto, in quanto le interrogazioni fatte sui tipi di buste speciali e normali sono veramente poche. Quindi risulta più semplice ed immediato interrogare direttamente l'entità busta e se si desidera capire la tipologia di busta si interroga l'attributo "is_speciale" (che sarà true oppure false).

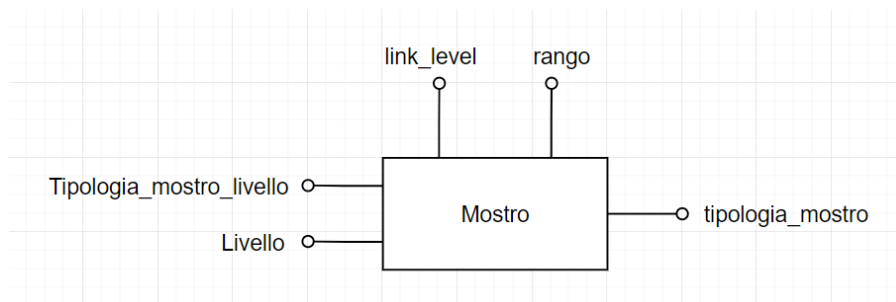


Non essendoci molti attributi delle entità figlie e non dovendo fare molte interrogazioni su di esse, non si sono trovate motivazioni per non applicare questo tipo di metodo. Essendo la gerarchia di tipo totale ed esclusiva non sono state apportate ulteriori modifiche. La chiave di busta è stata mantenuta ed è stato aggiunto l'attributo di selezione.

1.3 Gestione gerarchia mostro

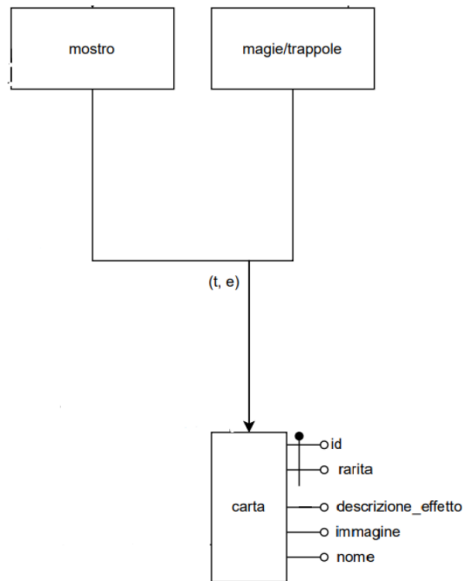


Come per l'entità busta, anche in “mostro” abbiamo una gerarchia di tipo totale ed esclusiva e non verranno effettuate molte interrogazioni sulle tabelle che ereditano da mostro. Esse servono per un fattore di classificazione, separazione e semplificazione del meccanismo di ricerca delle carte all'interno del database e per questo si è deciso di applicare un collasso verso l'alto anche per il fatto che le tabelle figlie non hanno molti attributi e in questo modo si potrà evitare di dover esplorare ulteriori tabelle per ottenere informazioni utili, ma si potrà direttamente interrogare prima di tutto l'attributo di selezione, **tipologia_mostro**, che ci dirà in quale entità “figlia” si ricade.

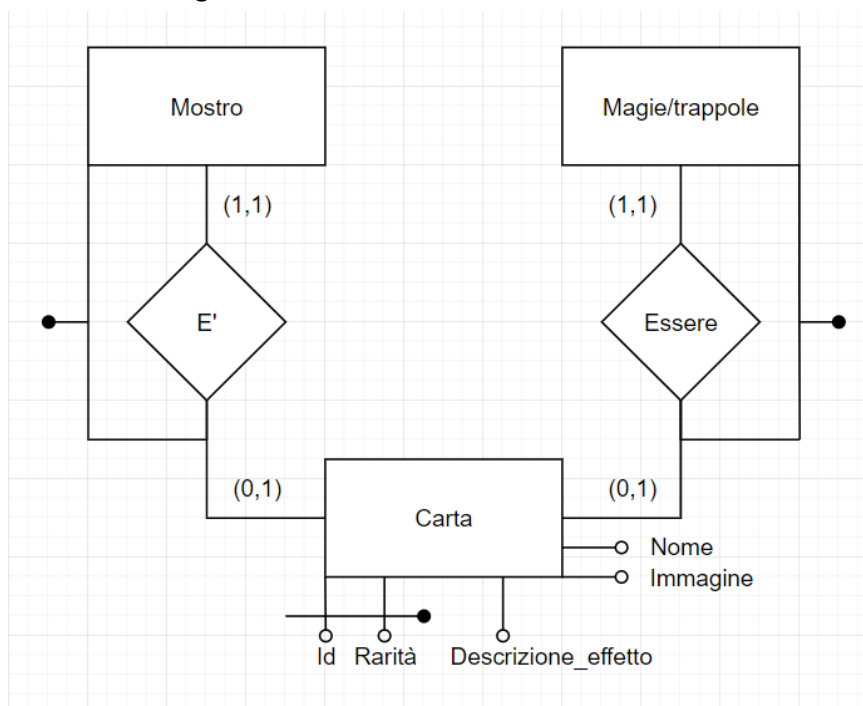


A seguito della scelta di applicare il collasso verso l'alto, l'entità “mostro” ha quindi ereditato tutti gli attributi delle entità figlie, dove “**tipologia_mostro_livello**” sarà un dato di tipo enum per esplorare l'ulteriore grado di particolarità dei mostri aventi un livello. Essendo che ogni entità figlia possiede un attributo ciascuna non sono stati trovati particolari problemi nell'applicare questo tipo di collasso.

1.4 Gestione gerarchia carta



Riguardo l'entità carta, il numero di interrogazioni verso le entità figlie, "mostro" e "magie/trappole", sono elevate e anche il numero di attributi sia nelle entità figlie che nella entità carta non è ridotto, allora si è deciso di utilizzare la metodologia del mantenimento utilizzando una chiave esterna all'interno delle entità figlie.

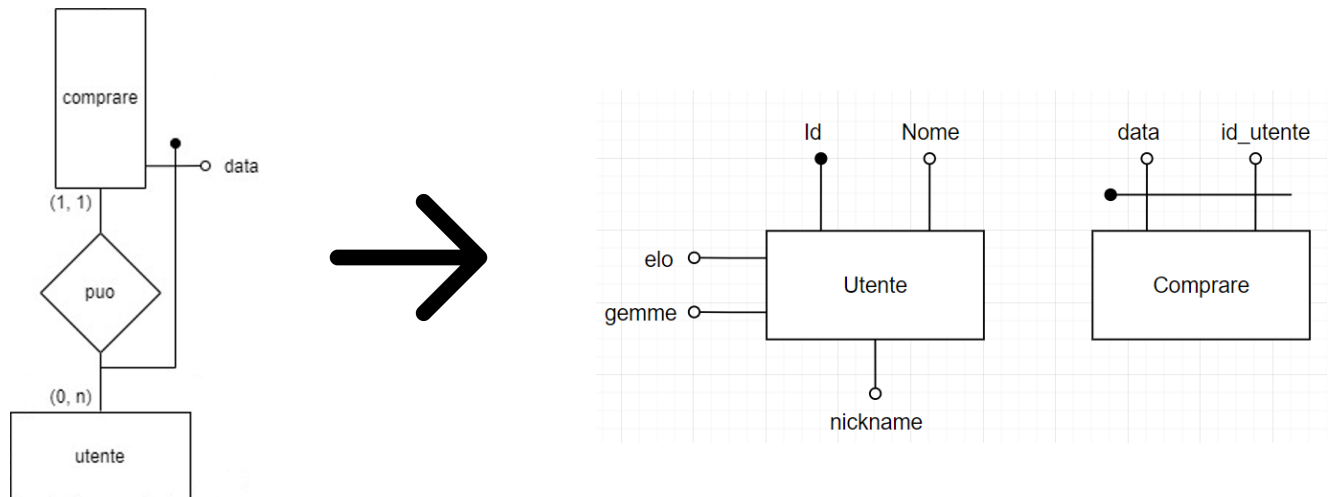


Questo tipo di eliminazione della gerarchia ci sembrava il più adatto in quanto, come detto prima, il numero di interrogazioni può essere fatto sia sull'entità più generale carta, ma anche sulle entità figlie per controllare gli attributi dell'entità mostro; in più gli attributi di questa entità non sono pochi e se si effettuasse un collasso verso l'alto ci sarebbe un overload inutile di attributi all'interno dell'entità carta. Il collasso verso il basso è stato evitato in quanto la frequenza di interrogazione dell'entità carta è elevata. Essendo la gerarchia di tipo totale ed esclusiva non sono state apportate ulteriori modifiche.

2. Selezione delle chiavi primarie ed eliminazione delle identificazioni esterne

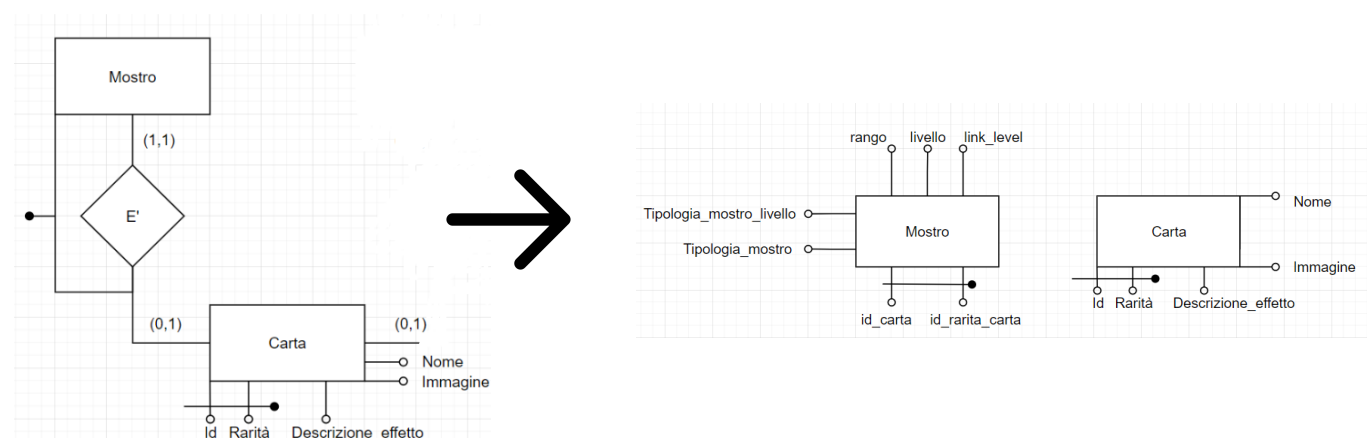
2.1 Entità comprare

L'entità comprare è ora identificata dal suo attributo "data" insieme ad un attributo id_utente che rappresenta la chiave primaria dell'utente a cui è riferito l'acquisto.



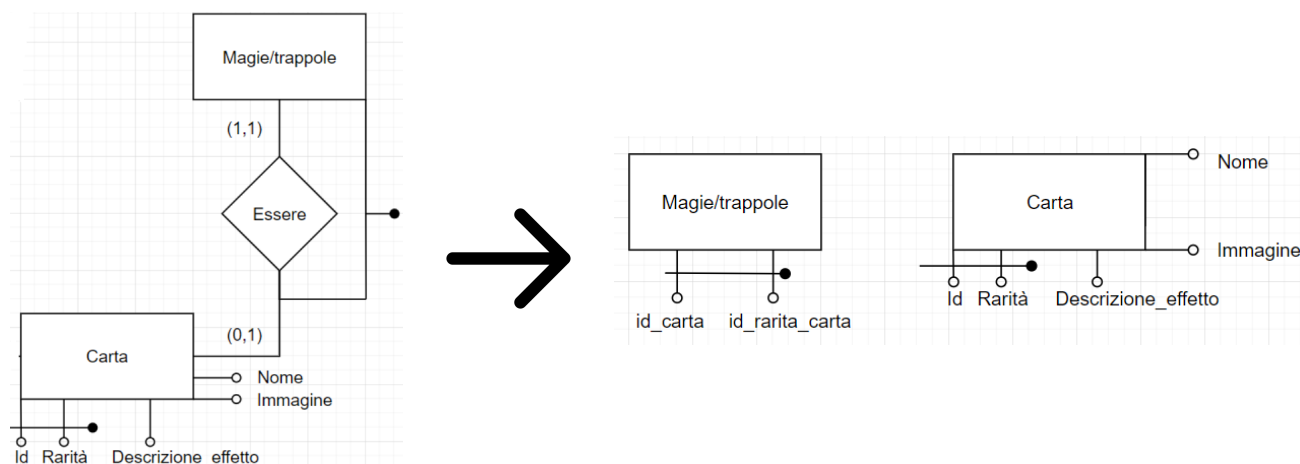
2.2 Entità mostro

L'entità mostro, avendo subito un'eliminazione della gerarchia tramite un mantenimento, ha riportato al suo interno una chiave esterna che si riferisce all'entità carta. Questa è stata eliminata ottenendo una chiave composta tra le chiavi dell'entità carta.



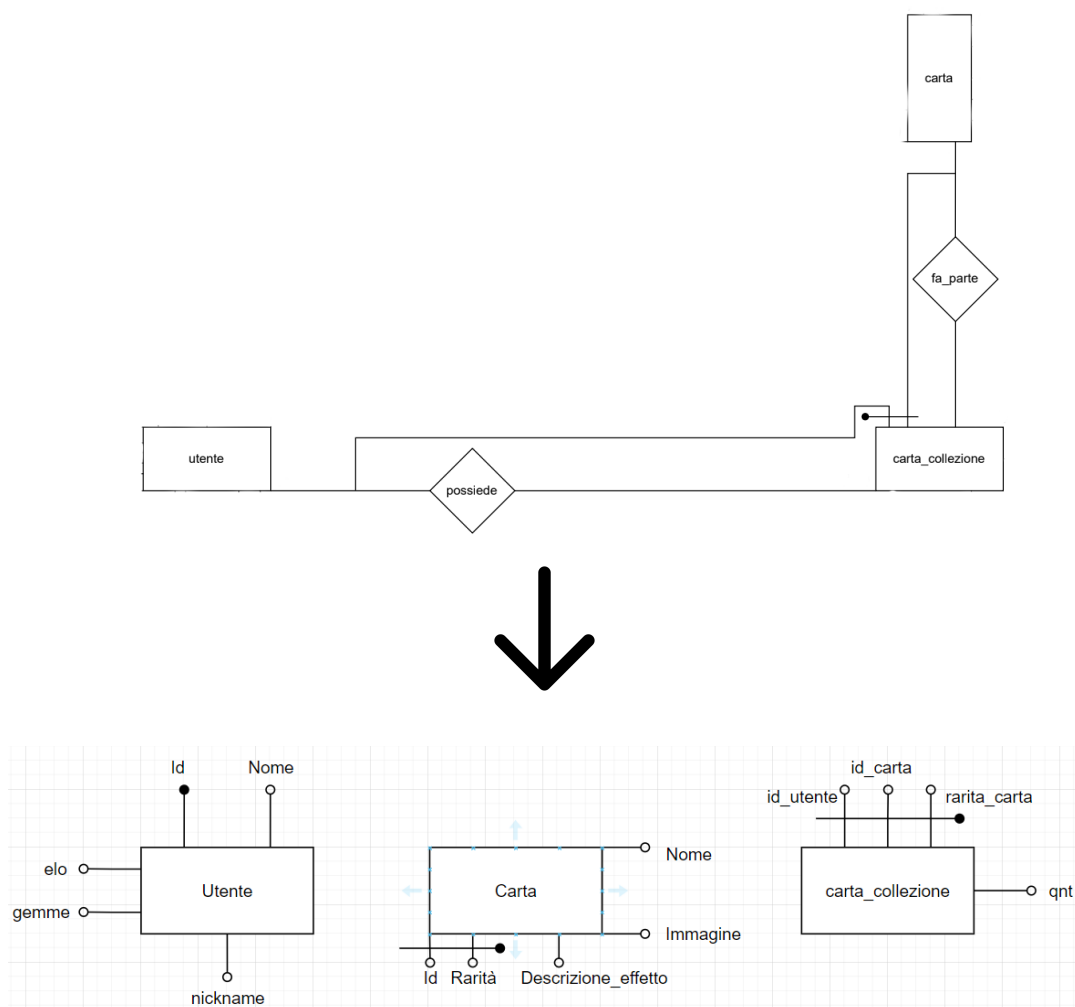
2.3 Entità magie/trappole

Come per l'entità mostro (sezione 2.2), anche per “magie/trappole” è stato applicato un mantenimento e perciò ha riportato una chiave esterna che si riferisce all'entità carta, formando una chiave composta.



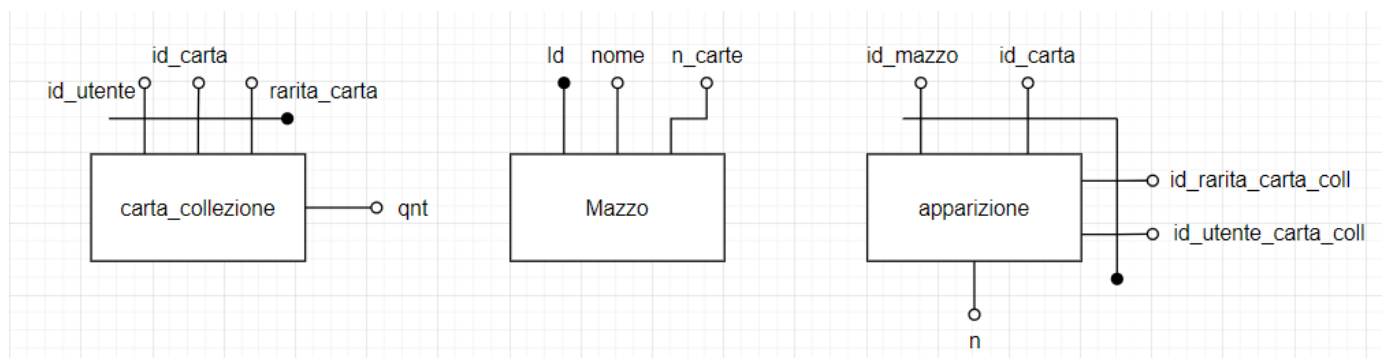
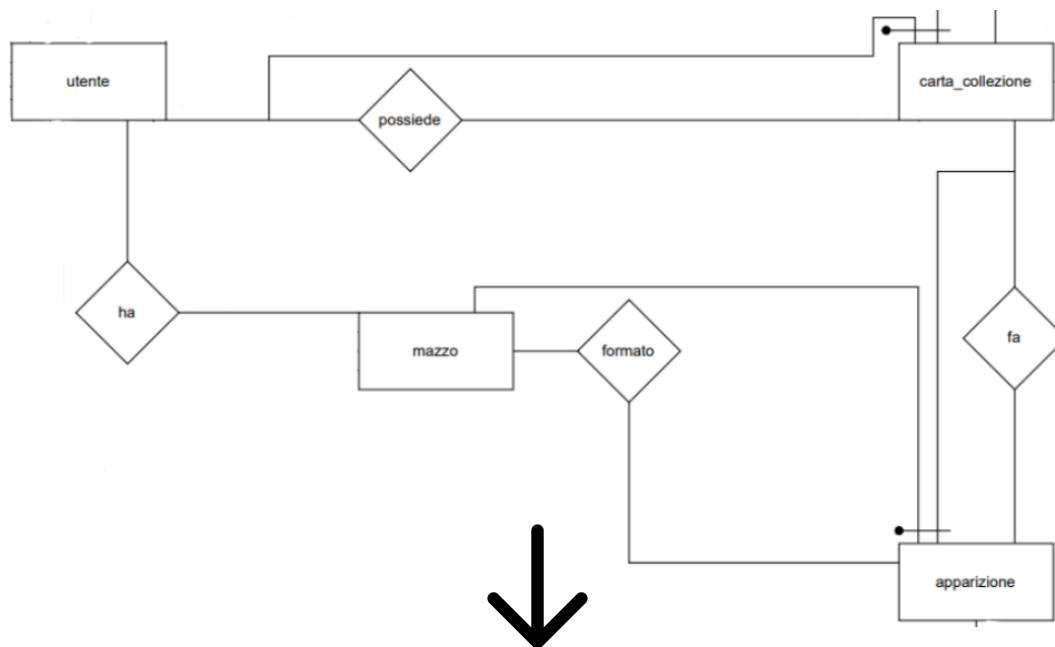
2.4 Entità carta_collezione

L'entità carta_collezione, ottenuta tramite una reificazione, possiede ovviamente delle chiavi esterne derivanti dalle sue tabelle adiacenti: esse sono state eliminate creando una chiave composta tra la chiave di “utente” e la chiave a sua volta composta dell'entità “carta”



2.5 Entità apparizione

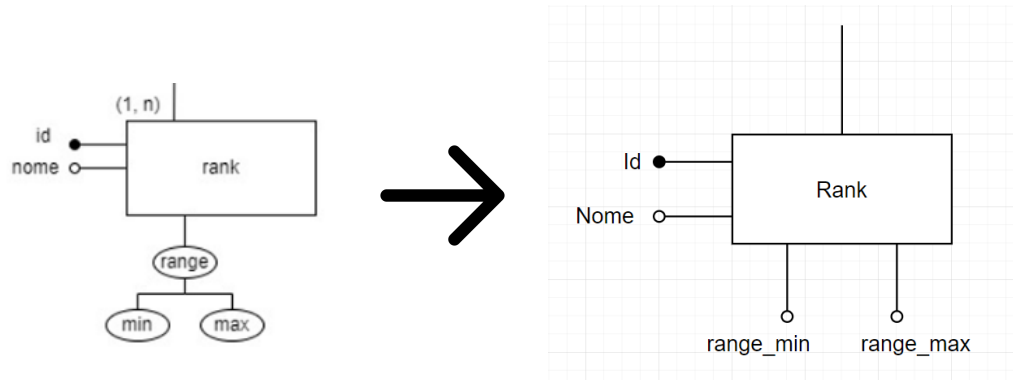
L'entità apparizione è stata ottenuta tramite un'ulteriore reificazione e per questo si sono eliminate le sue chiavi esterne creando una chiave composta tra le chiavi della tabella mazzo e quelle dell'entità "carta_collezione", quindi "apparizione" sarà identificata dall'id del mazzo a cui appartiene, dall'id della carta a cui fa riferimento, all'id della rarità della carta in questione e dall'id dell'utente a cui appartiene la carta, formando una chiave composta.



3. Trasformazione degli attributi composti o multipli

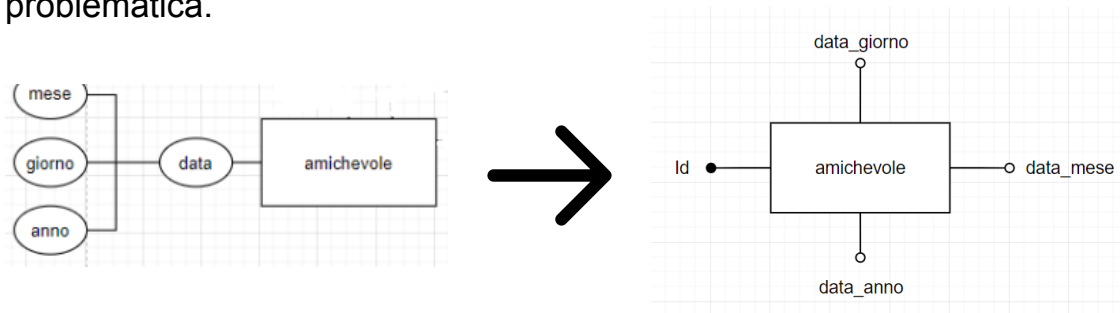
3.1 Trasformazione attributi dell'entità rank

L'entità "rank" possiede un attributo composto comprendente gli estremi dell'intervallo del rank stesso. Non essendoci vantaggi nella formulazione di un'entità aggiuntiva si è scelto di implementare due attributi atomici "range_min" e "range_max" che permetteranno un cambio automatico del rank di un utente tramite una gestione con trigger.



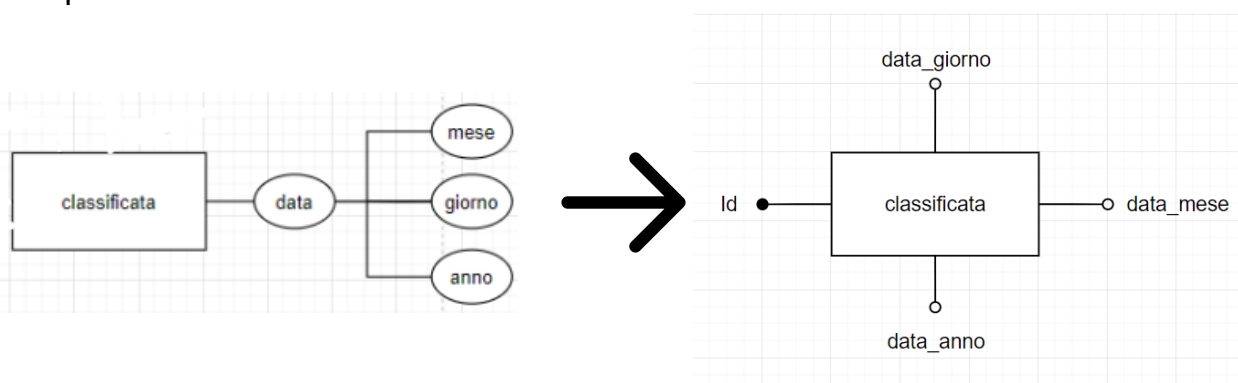
3.2 Trasformazione attributi dell'entità partita amichevole

L'attributo composto "data" appartenente all'entità "partita" è stato trasferito alle entità figlie "amichevole" e "classificata" per l'operazione di collasso verso il basso svolta per eliminare la gerarchia fra le entità. Per questo si è atomizzato l'attributo "data" in 3 attributi consoni. A livello di implementazione si è scelto però di utilizzare un attributo di tipo "date" per evitare ogni sorta di problematica.



3.3 Trasformazione attributi dell'entità partita classificata

Come per l'entità "amichevole" si è scelto di atomizzare l'attributo data in tre attributi. A livello implementativo si è sempre scelto di utilizzare un attributo di tipo "date".

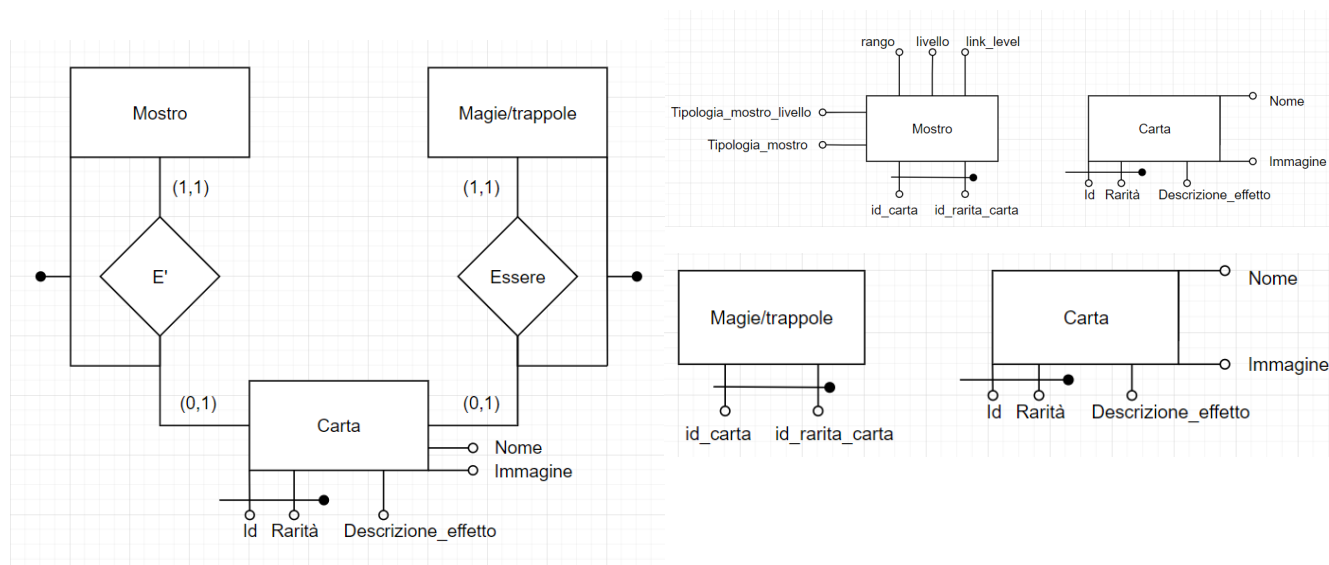


4. Traduzione di entità e associazioni in schemi di relazioni

In questa sezione si andranno a tradurre i principali pezzi del nostro database in schemi relazionali

- Memorizzazione, separazione e classificazione delle carte
- Descrizione del meccanismo di acquisto tramite buste
- Collezione di un utente, gestione dei mazzi e mercato
- Utenti, classificazione e partite amichevoli e classificate

4.1 Memorizzazione, separazione e classificazione delle carte



carta (id, id_rarita, nome, descrizione_effetto, immagine)

FK: id_rarita REFERENCES rarita

l'attributo rarità è stato trasformato in un'entità per una maggiore correttezza e solidità dei dati.

rarita (id, nome)

magie_trappole (id_carta, id_rarita_carta, id_categoria)

FK: id_carta REFERENCES carta

FK: id_rarita_carta REFERENCES carta

FK: id_categoria REFERENCES categoria

L'entità "categoria" non è presente nelle immagini in questa sezione (4.1) ma può essere vista nel modello E/R completo o nelle opportune sezioni.

categoria (id, nome)

mostro(id_carta, id_rarita_carta, tipologia_mostro, tipologia_mostro_livello, link_level, rango, id_sotto_tipo, id_attributo, livello)

FK: id_carta **REFERENCES** carta

FK: id_rarita_carta **REFERENCES** carta

FK: id_sotto_tipo **REFERENCES** sotto_tipo

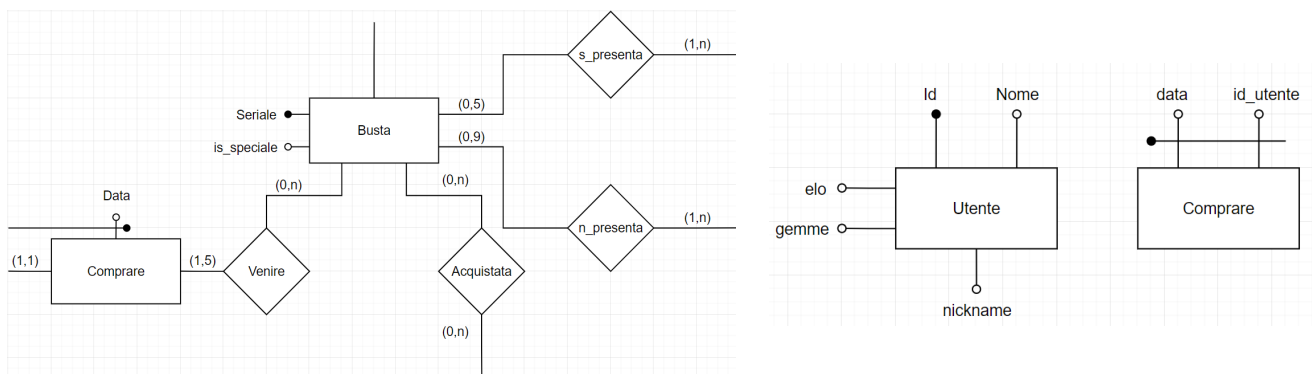
FK: id_attributo **REFERENCES** attributo

attributo (id, nome)

sotto_tipo (id, nome)

4.2 Descrizione del meccanismo di acquisto tramite buste

Le entità non visibili nella seguente immagine ma presenti negli schemi di relazioni successivi sono visibili nelle opportune sezioni o nel modello E/R completo.



le associazioni di tipo molti a molti sono state tradotte in entità.

espansione (id, nome, anno)

compare (id_espansione, id carta, id rarita carta)

FK: id_espansione **REFERENCES** espansione

FK: id_carta **REFERENCES** carta

FK: id_rarita_carta **REFERENCES** carta

busta (seriale, id_espansione, is_speciale)

FK: id_espansione **REFERENCES** espansione

venire (seriale_busta, id_utente_comprare, data_comprare)

FK: seriale_busta **REFERENCES** busta

FK: id_utente_comprare **REFERENCES** comprare

FK: data_comprare **REFERENCES** comprare

comprare (data, id_utente)

FK: id_utente REFERENCES utente

s_presenta (seriale, id_carta, id_rarita_carta)

FK: seriale REFERENCES busta

FK: id_carta REFERENCES carta

FK: id_rarita_carta REFERENCES carta

n_presenta (seriale, id_carta, id_rarita_carta)

FK: seriale REFERENCES busta

FK: id_carta REFERENCES carta

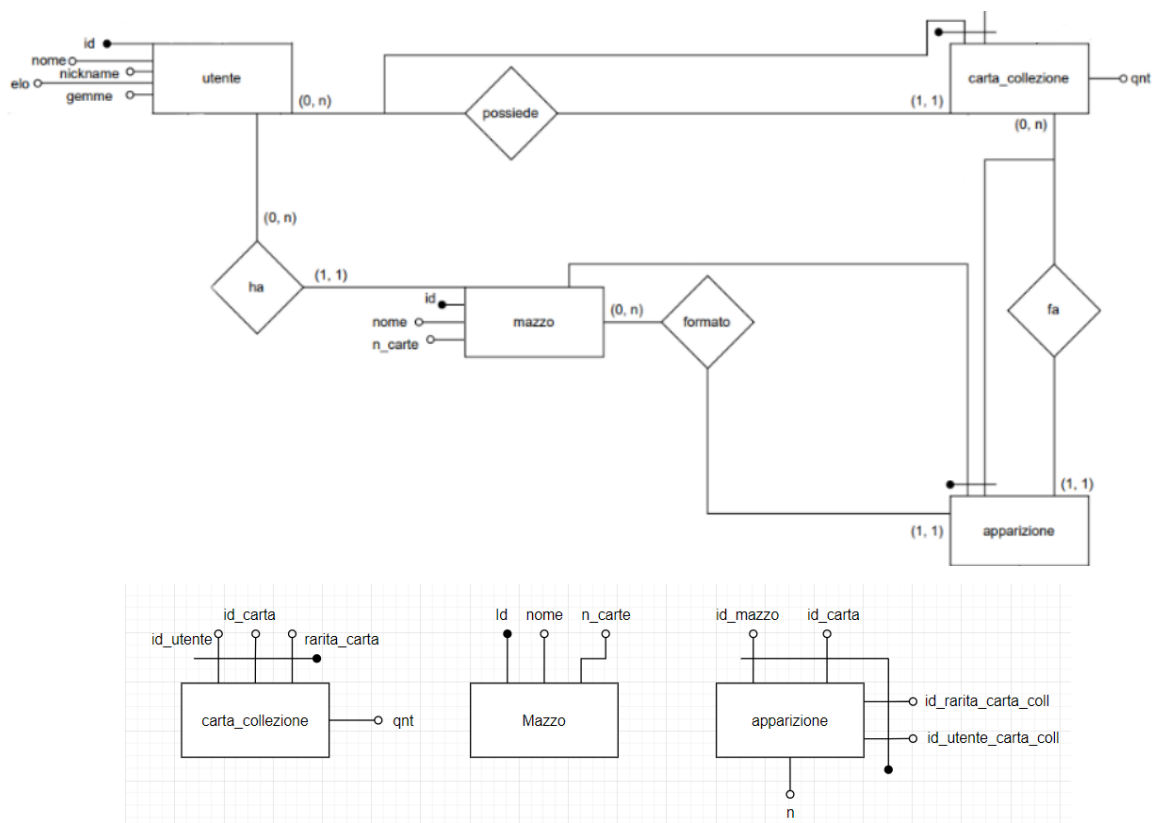
FK: id_rarita_carta REFERENCES carta

acquistata (seriale, id_utente)

FK: seriale REFERENCES busta

FK: id_utente REFERENCES utente

4.3 Collezione di un utente e gestione dei mazzi e del mercato



mazzo (id, nome, n_carte, id_utente)

FK: id_utente REFERENCES utente

carta_collezione (id_utente, id_carta, id_rarita_carta, qnt)

FK: id_utente REFERENCES utente

FK: id_carta REFERENCES carta

FK: id_rarita_carta REFERENCES carta

L'attributo "n" di apparizione apparteneva all'entità copia prima di essere stata eliminata per un'efficiente implementazione.

apparizione (id_mazzo, id_utente_carta_coll,
id_carta_carta_coll, id_rarita_carta_coll, n)

FK: id_mazzo **REFERENCES** mazzo

FK: id_utente_carta_coll **REFERENCES** carta_collezione

FK: id_carta_carta_coll **REFERENCES** carta_collezione

FK: id_rarita_carta_coll **REFERENCES** carta_collezione

mercato(id_utente, id_utente_carta_coll,
id_carta_carta_coll, id_rarita_carta_coll, qnt, prezzo)

FK: id_utente **REFERENCES** utente

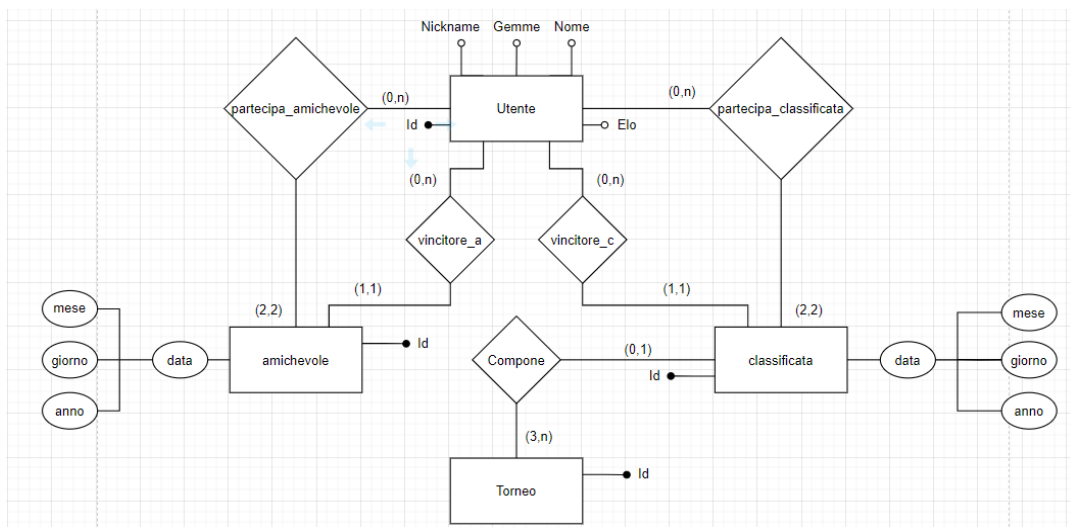
FK: id_utente_carta_coll **REFERENCES** carta_collezione

FK: id_carta_carta_coll **REFERENCES** carta_collezione

FK: id_rarita_carta_coll **REFERENCES** carta_collezione

4.4 Utenti, classificazione e partite amichevoli e classificate

Le entità non visibili nella seguente immagine ma presenti negli schemi di relazioni successivi sono visibili nelle opportune sezioni o nel modello E/R completo.



Le associazioni di tipo molti a molti sono state tradotte in entità.

utente (id, nome, nickname, elo, gemme, id_rank)

FK: id_rank **REFERENCES** rank

rank (id, nome, range_min, range_max)

amicizia (id_utente_1, id_utente_2)

FK: id_utente_1 **REFERENCES** utente

FK: id_utente_2 **REFERENCES** utente

partecipa_amichevole (id_utente, id_amichevole)

FK: id_utente **REFERENCES** utente

FK: id_amichevole **REFERENCES** amichevole

partecipa_classificata (id_utente, id_classificata)

FK: id_utente **REFERENCES** utente

FK: id_classificata **REFERENCES** classificata

amichevole(id, id_utente_vincitore, data_giorno,
data_mese, data_anno)

FK: id_utente_vincitore **REFERENCES** utente

classificata(id, premio, id_torneo, id_utente_vincitore,
data_giorno, data_mese, data_anno)

FK: id_utente_vincitore **REFERENCES** utente

FK: id_torneo **REFERENCES** torneo

torneo (id)

5. Verifica di normalizzazione

La normalizzazione è il processo di organizzazione dei dati in un database. Tale processo comprende la creazione di tabelle e la definizione di relazioni tra di esse sulla base di regole progettate in modo da proteggere i dati e rendere il database più flessibile mediante l'eliminazione della ridondanza e delle dipendenze incoerenti.

Dopo aver effettuato la verifica della normalizzazione non abbiamo ritenuto necessaria alcuna ulteriore modifica.

Punto D, operazioni previste dalla base di dati – Descrizione e relativo codice SQL

1. Query di selezione

Dopo ogni query di selezione sono presenti immagini sul relativo output.

1.1 Seleziona tutte le partite classificate vinte dell'utente

```
SELECT
    u.nickname AS vincitore, u.elo, c.data, c.premio,
    (u.elo+c.premio) AS nuovo_elo
FROM
    utente AS u, classificata AS c
WHERE
    c.id_utente_vincitore=u.id AND u.nickname='Fede';
```

	vincitore character varying (255)	elo integer	data date	premio integer	nuovo_elo integer
1	Fede	500	2020-12-15	500	1000
2	Fede	500	2022-01-28	1000	1500

1.2 Selezione di tutte le carte con le relative apparizioni e possessore in un determinato mazzo

Come esempio qui si è scelto di prendere in esame il mazzo chiamato “EROE Elementale” e come utente possessore, “Giacco”.

```
SELECT
    u.nickname AS possessore, c.nome AS nome_carta,
    r.nome AS rarita_carta, a.n AS quantita
FROM
    carta AS c, rarita AS r, mazzo AS m, apparizioni AS
a, carta_collezione AS cc, utente AS u
WHERE
    u.id=cc.id_utente AND cc.id_carta=c.id AND
    cc.id_rarita_carta=c.id_rarita AND
    c.id_rarita=r.id AND
    a.id_utente_carta_coll=cc.id_utente AND
    a.id_carta_carta_coll=cc.id_carta AND
    a.id_rarita_carta_coll=cc.id_rarita_carta AND
    m.nome='EROE Elementale' AND u.nickname='Giacco'
GROUP BY
    u.nickname, c.nome, r.nome, a.n
```

	possessore character varying (255) 🔒	nome_carta character varying (64) 🔒	rarita_carta character varying (64) 🔒	quantita integer 🔒
1	Giacco	Yubel	Ultra	1
2	Giacco	Elemental HERO Solid Soldier	Ultimate	3
3	Giacco	Elemental HERO Honest Neos	Ultra	1
4	Giacco	Elemental HERO Neos	Ultra	1
5	Giacco	Elemental HERO Spirit of Neos	Ultra	1
6	Giacco	Elemental HERO Neos Alius	Ultra	2

1.3 Selezione di tutti gli utenti che non hanno mai vinto una partita classificata

```

SELECT
    u.nome, u.nickname, r.nome AS rank, u.elo, u.gemme
FROM
    utente AS u, rank AS r
WHERE u.id_rank=r.id AND NOT EXISTS
    (SELECT *
    FROM classificata AS c
    WHERE c.id_utente_vincitore=u.id);

```

	nome character varying (64) 🔒	nickname character varying (255) 🔒	rank character varying (64) 🔒	elo integer 🔒	gemme integer 🔒
1	Gabriele	Cuso	rookie	180	156

1.4 Seleziona tutte le carte che sono di tipo 'mostro_effetto', non sono di sotto_tipo=normale e non hanno attributo=luce (query di ricerca generica)

```

SELECT
    c.nome AS nome_carta, st.nome AS sotto_tipo, a.nome
AS attributo, r.nome AS rarita_carta
FROM
    carta AS c, rarita AS r, mostro AS m, sotto_tipo AS
st, attributo AS a
WHERE
    c.id=m.id_carta AND c.id_rarita=r.id AND
    m.id_sotto_tipo=st.id AND m.id_attributo=a.id AND
    m.tipologia_mostro_livello='mostro_effetto' AND
    NOT EXISTS
    (SELECT *
    FROM sotto_tipo AS st2

```

```

WHERE
    m.id_sotto_tipo=st2.id AND st2.nome='normale'
) AND NOT EXISTS
(SELECT *
FROM attributo AS a2
WHERE
    m.id_attributo=a2.id AND a2.nome='luce'
);

```

	nome_carta character varying (64) 🔒	sotto_tipo character varying (255) 🔒	attributo character varying (255) 🔒	rarity_carta character varying (64) 🔒
1	Elemental HERO Honest Neos	guerriero	vento	Ultra

1.5 Seleziona tutti gli utenti che hanno lo stesso rank dell'utente 'Fede' (query di matchmaking)

```

SELECT
    u.nome, u.nickname, r.nome AS rank, u.elo
FROM
    utente AS u, rank AS r
WHERE
    u.id_rank=r.id AND u.nickname<>'Fede' AND
    u.id_rank=(SELECT u2.id_rank
                FROM utente AS u2
                WHERE u2.nickname='Fede');

```

	nome character varying (64) 🔒	nickname character varying (255) 🔒	rank character varying (64) 🔒	elo integer 🔒
1	Gabriele	Cuso	bronze	680

1.6 Seleziona tutte le carte presenti nel mercato che presentano nel nome la parola 'HERO', attributo=oscurità e non sono immesse nel mercato dall'utente che sta ricercando la carta cioè 'Fede'

```

SELECT
    u.nickname AS venditore, c.nome AS nome_carta,
    st.nome AS sotto_tipo, a.nome AS attributo, r.nome AS
    rarity, m.qnt AS disponibilita, m.prezzo
FROM
    mercato AS m, utente AS u, carta_collezione AS cc,
    carta AS c, rarity AS r, mostro AS ms, attributo AS a,
    sotto_tipo AS st

```

WHERE

```
u.id=m.id_utente AND c.id_rarita=r.id AND
c.id=cc.id_carta AND c.id_rarita=cc.id_rarita_carta
AND
ms.id_carta=c.id AND ms.id_rarita_carta=c.id_rarita
AND
ms.id_sotto_tipo=st.id AND ms.id_attributo=a.id AND
u.id=cc.id_utente AND
m.id_utente_carta_coll=cc.id_utente AND
m.id_carta_carta_coll=cc.id_carta AND
m.id_rarita_carta_coll=cc.id_rarita_carta AND
c.nome LIKE '%HERO%' AND a.nome='oscurità' AND
u.nickname<>'Fede';
```

	venditore character varying (255)	nome_carta character varying (64)	sotto_tipo character varying (255)	attributo character varying (255)	rarita character varying (64)	disponibilita integer	prezzo character varying (64)
1	Giacco	Elemental HERO Solid Soldier	guerriero	oscurità	Ultimate	1	50

2. Query di modifica

Queste query di modifica permettono il test di alcuni dei trigger elencati nel punto extra A. L'immagine sottostante ad ogni query riporta il corretto funzionamento della procedura. Le istruzioni complete per il test di ogni singolo trigger sono riportate nei file update.sql e trigger_test.sql presenti nella cartella di consegna.

UPDATE apparizioni SET n=4 WHERE id_carta_carta_coll=1

```
ERROR: ERRORE: Maximum limit for the same card reached
CONTEXT:  funzione PL/pgSQL limite_3_carte_mazzo_update() riga 4 a RAISE

SQL state: P0001
```

UPDATE utente SET elo=680 WHERE nickname='Cuso';

	id [PK] bigint	id_rank integer	nome character varying (64)	nickname character varying (255)	elo integer	gemme integer
1	1	2	Federico	Fede	500	0
2	3	4	Alessandro	Giacco	1456	862
3	2	2	Gabriele	Cuso	680	156

UPDATE mazzo SET n_carte=4 WHERE nome='Fantasmatrucco';

```
ERROR: ERRORE: Minimum limit of cards in the deck is 40
CONTEXT:  funzione PL/pgSQL limite_carte_update() riga 4 a RAISE
```

```
SQL state: P0001
```

3. Query di eliminazione (per mostrare il DELETE CASCADE)

Situazione prima della DELETE:

tabella comprare

	data [PK] date	id_utente [PK] integer
1	2019-05-18	3
2	2019-12-25	3
3	2022-07-30	3
4	2022-07-29	3
5	2022-07-28	3
6	2022-07-27	3
7	2022-07-26	3

tabella venire

	seriale_busta [PK] character varying (255)	id_utente_comprare [PK] integer	data_comprare [PK] date
1	CHACHA2000	3	2019-05-18
2	CHACHA2000	3	2019-12-25
3	CHACHA2000	3	2022-07-30
4	CHACHA2000	3	2022-07-29
5	CHACHA2000	3	2022-07-28
6	CHACHA2000	3	2022-07-27
7	CHACHA2000	3	2022-07-26

DELETE FROM comprare WHERE data='2022-07-26';

Situazione dopo la DELETE

	data [PK] date	id_utente [PK] integer
1	2019-05-18	3
2	2019-12-25	3
3	2022-07-30	3
4	2022-07-29	3
5	2022-07-28	3
6	2022-07-27	3

	seriale_busta [PK] character varying (255)	id_utente_comprare [PK] integer	data_comprare [PK] date
1	CHACHA2000	3	2019-05-18
2	CHACHA2000	3	2019-12-25
3	CHACHA2000	3	2022-07-30
4	CHACHA2000	3	2022-07-29
5	CHACHA2000	3	2022-07-28
6	CHACHA2000	3	2022-07-27

Punto E, codice SQL per la creazione delle tabelle e per l'inserimento dei dati

Si presentano le query volte alla creazione delle tabelle. Ogni tabella è stata creata con il parametro IF NOT EXIST che ci permette di non creare errore nel caso in cui la tabella esista già. La motivazione va ricercata in un contesto di scalabilità e modularità che si è cercato di dare al progetto. Seguendo sempre la logica di modularità è inoltre stata divisa la creazione delle tabelle dalla creazione delle chiavi UNIQUE, FOREIGN KEY e PRIMARY KEY. Le FOREIGN KEY sono state gestite a livello di database attraverso i costrutti ON DELETE e ON UPDATE. Per quanto riguarda le PRIMARY KEY è stato utilizzato il tipo di dato BIGSERIAL(intero autoincrementante).

Creazione tabelle

```
CREATE TABLE IF NOT EXISTS rank (  
    id BIGSERIAL NOT NULL,  
    nome varchar(64) NOT NULL,  
    range_min integer NOT NULL,  
    range_max integer NOT NULL  
);
```

```
ALTER TABLE rank  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_rank UNIQUE (nome, range_min,  
range_max);
```

2. Creazione tabella utente

```
CREATE TABLE IF NOT EXISTS utente (  
    id BIGSERIAL NOT NULL,  
    id_rank integer NOT NULL,  
    nome varchar(64) DEFAULT NULL,  
    nickname varchar(255) NOT NULL,  
    elo integer DEFAULT NULL,  
    gemme integer DEFAULT NULL  
);
```

```
ALTER TABLE utente  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_utente UNIQUE (nickname),  
    ADD CONSTRAINT fk_rank FOREIGN KEY (id_rank)  
REFERENCES rank(id) ON DELETE CASCADE;
```


Attraverso un trigger viene gestito l'id del rank in base al valore all'interno della colonna elo

3. Creazione tabella amicizia

```
CREATE TABLE IF NOT EXISTS amicizia (  
    id_utente_1 integer NOT NULL,  
    id_utente_2 integer NOT NULL  
);  
  
ALTER TABLE amicizia  
    ADD PRIMARY KEY (id_utente_1, id_utente_2),  
    ADD CONSTRAINT fk_amicizia_1 FOREIGN KEY  
(id_utente_1) REFERENCES utente(id) ON UPDATE CASCADE ON  
DELETE CASCADE,  
    ADD CONSTRAINT fk_amicizia_2 FOREIGN KEY  
(id_utente_2) REFERENCES utente(id) ON UPDATE CASCADE ON  
DELETE CASCADE;
```

4. Creazione tabella amichevole

```
CREATE TABLE IF NOT EXISTS amichevole (  
    id BIGSERIAL NOT NULL,  
    id_utente_vincitore integer NOT NULL,  
    data date DEFAULT NULL  
);  
  
ALTER TABLE amichevole  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT fk_amichevole_1 FOREIGN KEY  
(id_utente_vincitore) REFERENCES utente(id) ON UPDATE  
CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT unica_amichevole UNIQUE  
(id_utente_vincitore, data);
```

5. Creazione tabella torneo

```
CREATE TABLE IF NOT EXISTS torneo (  
    id integer NOT NULL  
);
```

```
ALTER TABLE torneo  
    ADD PRIMARY KEY (id);
```

6. Creazione tabella classificata

```
CREATE TABLE IF NOT EXISTS classificata (  
    id BIGSERIAL NOT NULL,  
    id_utente_vincitore integer NOT NULL,  
    id_torneo integer NOT NULL,  
    data date DEFAULT NULL,  
    premio integer DEFAULT NULL  
);
```

```
ALTER TABLE classificata  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT fk_classificata_1 FOREIGN KEY  
(id_utente_vincitore) REFERENCES utente(id) ON UPDATE  
CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_classificata_2 FOREIGN KEY  
(id_torneo) REFERENCES torneo(id) ON UPDATE CASCADE ON  
DELETE CASCADE,  
    ADD CONSTRAINT unica_classificata UNIQUE  
(id_utente_vincitore, data);
```

7. Creazione tabella *partecipa_amichevole*

```
CREATE TABLE IF NOT EXISTS partecipa_amichevole (  
    id_utente integer NOT NULL,  
    id_amichevole integer DEFAULT NULL  
);
```

```
ALTER TABLE partecipa_amichevole  
    ADD PRIMARY KEY (id_utente, id_amichevole),  
    ADD CONSTRAINT fk_partecipa_amichevole_1 FOREIGN KEY  
(id_utente) REFERENCES utente(id) ON UPDATE CASCADE ON  
DELETE CASCADE,  
    ADD CONSTRAINT fk_partecipa_amichevole_2 FOREIGN KEY  
(id_amichevole) REFERENCES amichevole(id) ON UPDATE  
CASCADE ON DELETE CASCADE;
```

8. Creazione tabella *partecipa_classificata*

```
CREATE TABLE IF NOT EXISTS partecipa_classificata (  
    id_utente integer NOT NULL,  
    id_classificata integer DEFAULT NULL  
);
```

```
ALTER TABLE partecipa_classificata  
    ADD PRIMARY KEY (id_utente, id_classificata),  
    ADD CONSTRAINT fk_partecipa_amichevole_1 FOREIGN KEY  
(id_utente) REFERENCES utente(id) ON UPDATE CASCADE ON  
DELETE CASCADE,  
    ADD CONSTRAINT fk_partecipa_amichevole_2 FOREIGN KEY  
(id_classificata) REFERENCES classificata(id) ON UPDATE  
CASCADE ON DELETE CASCADE;
```

9. Creazione tabella mazzo

```
CREATE TABLE IF NOT EXISTS mazzo (  
    id BIGSERIAL NOT NULL,  
    id_utente integer NOT NULL,  
    nome varchar(255) NOT NULL,  
    n_carte integer DEFAULT NULL  
);
```

```
ALTER TABLE mazzo  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT fk_mazzo_1 FOREIGN KEY (id_utente)  
REFERENCES utente(id) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT unica_mazzo UNIQUE (id_utente, nome);
```

Attraverso un trigger viene gestito il numero massimo di carte che posso avere all'interno del mazzo (n_carte), il motivo per cui si è deciso di usare un trigger piuttosto che un semplice controllo attraverso il costrutto CHECK è perché attraverso il trigger si può impostare un valore predefinito se il valore inserito dall'utente non è corretto.

10. Creazione tabella rarita

```
CREATE TABLE IF NOT EXISTS rarita (  
    id BIGSERIAL NOT NULL,  
    nome varchar(64) NOT NULL  
);
```

```
ALTER TABLE rarita  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_rarita UNIQUE (nome);
```

11. Creazione tabella carta

```
CREATE TABLE IF NOT EXISTS carta (  
    id BIGSERIAL NOT NULL,  
    id_rarita integer NOT NULL,  
    nome varchar(64) NOT NULL,  
    descrizione_effetto varchar(255) DEFAULT NULL,  
    immagine varchar(255) DEFAULT NULL  
);  
  
ALTER TABLE carta  
    ADD PRIMARY KEY (id, id_rarita),  
    ADD CONSTRAINT fk_carta_1 FOREIGN KEY (id_rarita)  
REFERENCES rarita(id) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT unica_carta UNIQUE (id_rarita, nome);
```

12. Creazione tabella carta_collezione

```
CREATE TABLE IF NOT EXISTS carta_collezione (  
    id_utente integer NOT NULL,  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL,  
    qnt integer NOT NULL  
);  
  
ALTER TABLE carta_collezione  
    ADD PRIMARY KEY (id_utente, id_carta,  
id_rarita_carta),  
    ADD CONSTRAINT fk_carta_collezione_1 FOREIGN KEY  
(id_utente) REFERENCES utente(id) ON UPDATE CASCADE ON  
DELETE CASCADE,  
    ADD CONSTRAINT fk_carta_collezione_2 FOREIGN KEY  
(id_carta, id_rarita_carta) REFERENCES carta(id,  
id_rarita) ON UPDATE CASCADE ON DELETE CASCADE;
```

13. Creazione tabella mercato

```
CREATE TABLE IF NOT EXISTS mercato (  
    id_utente integer NOT NULL,  
    id_utente_carta_coll integer NOT NULL,  
    id_carta_carta_coll integer NOT NULL,  
    id_rarita_carta_coll integer NOT NULL,  
    qnt integer DEFAULT NULL,  
    prezzo varchar(64) DEFAULT NULL  
);  
  
ALTER TABLE mercato  
    ADD PRIMARY KEY (id_utente, id_utente_carta_coll,  
id_carta_carta_coll, id_rarita_carta_coll),  
    ADD CONSTRAINT fk_mercato_1 FOREIGN KEY (id_utente)  
REFERENCES utente(id) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT fk_mercato_2 FOREIGN KEY  
(id_utente_carta_coll, id_carta_carta_coll,  
id_rarita_carta_coll) REFERENCES  
carta_collezione(id_utente, id_carta, id_rarita_carta) ON  
UPDATE CASCADE ON DELETE CASCADE;
```

14. Creazione tabella apparizioni

```
CREATE TABLE IF NOT EXISTS apparizioni (  
    id_mazzo integer NOT NULL,  
    id_utente_carta_coll integer NOT NULL,  
    id_carta_carta_coll integer NOT NULL,  
    id_rarita_carta_coll integer NOT NULL,  
    n integer NOT NULL  
);  
  
ALTER TABLE apparizioni  
    ADD PRIMARY KEY (id_mazzo, id_utente_carta_coll,  
id_carta_carta_coll, id_rarita_carta_coll),  
    ADD CONSTRAINT fk_apparizioni_1 FOREIGN KEY  
(id_utente_carta_coll, id_carta_carta_coll,  
id_rarita_carta_coll) REFERENCES  
carta_collezione(id_utente, id_carta, id_rarita_carta) ON  
UPDATE CASCADE ON DELETE CASCADE;
```

Attraverso due trigger viene gestito il fatto di non poter avere la stessa carta più di 3 volte ($n > 3$) tenendo conto della rarità (concetto spiegato nell'opportuna sezione dove sono presenti i trigger).

15. Creazione tabella espansione

```
CREATE TABLE IF NOT EXISTS espansione (  
    id BIGSERIAL NOT NULL,  
    nome varchar(255) NOT NULL,  
    anno integer DEFAULT NULL  
);  
  
ALTER TABLE espansione  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_espansione UNIQUE (nome);
```

16. Creazione tabella compare

```
CREATE TABLE IF NOT EXISTS compare (  
    id_espansione integer NOT NULL,  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL  
);  
  
ALTER TABLE compare  
    ADD PRIMARY KEY (id_espansione, id_carta,  
    id_rarita_carta),  
    ADD CONSTRAINT fk_compare_1 FOREIGN KEY  
(id_espansione) REFERENCES espansione(id) ON UPDATE  
CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_compare_2 FOREIGN KEY (id_carta,  
    id_rarita_carta) REFERENCES carta(id, id_rarita) ON  
UPDATE CASCADE ON DELETE CASCADE;
```

17. Creazione tabella busta

```
CREATE TABLE IF NOT EXISTS busta (  
    seriale varchar(255) NOT NULL,  
    id_espansione integer NOT NULL,  
    is_speciale BOOLEAN NOT NULL  
);  
  
ALTER TABLE busta  
    ADD PRIMARY KEY (seriale),  
    ADD CONSTRAINT fk_busta_1 FOREIGN KEY (id_espansione)  
REFERENCES espansione(id) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT unica_busta UNIQUE (seriale,  
    id_espansione);
```


18. Creazione tabella comprare

```
CREATE TABLE IF NOT EXISTS comprare (  
    data date NOT NULL,  
    id_utente integer NOT NULL  
);  
  
ALTER TABLE comprare  
    ADD PRIMARY KEY (data, id_utente),  
    ADD CONSTRAINT fk_comprare_1 FOREIGN KEY (id_utente)  
REFERENCES utente(id) ON UPDATE CASCADE ON DELETE  
CASCADE;
```

19. Creazione tabella venire

```
CREATE TABLE IF NOT EXISTS venire (  
    seriale_busta varchar(255) NOT NULL,  
    id_utente_comprare integer NOT NULL,  
    data_comprare date NOT NULL  
);  
  
ALTER TABLE venire  
    ADD PRIMARY KEY (seriale_busta, id_utente_comprare,  
data_comprare),  
    ADD CONSTRAINT fk_venire_1 FOREIGN KEY  
(seriale_busta) REFERENCES busta(seriale) ON UPDATE  
CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_venire_2 FOREIGN KEY  
(id_utente_comprare, data_comprare) REFERENCES  
comprare(id_utente, data) ON UPDATE CASCADE ON DELETE  
CASCADE;
```

20. Creazione tabella s_presenta

```
CREATE TABLE IF NOT EXISTS s_presenta (  
    seriale varchar(255) NOT NULL,  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL  
);  
  
ALTER TABLE s_presenta  
    ADD PRIMARY KEY (seriale, id_carta, id_rarita_carta),  
    ADD CONSTRAINT fk_s_presenta_1 FOREIGN KEY (seriale)  
REFERENCES busta(seriale) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT fk_s_presenta_2 FOREIGN KEY (id_carta,  
id_rarita_carta) REFERENCES carta(id, id_rarita) ON  
UPDATE CASCADE ON DELETE CASCADE;
```

21. Creazione tabella acquistata

```
CREATE TABLE IF NOT EXISTS acquistata (  
    seriale varchar(255) NOT NULL,  
    id_utente integer NOT NULL  
);  
  
ALTER TABLE acquistata  
    ADD PRIMARY KEY (seriale, id_utente),  
    ADD CONSTRAINT fk_acquista_1 FOREIGN KEY (seriale)  
REFERENCES busta(seriale) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT fk_acquista_2 FOREIGN KEY (id_utente)  
REFERENCES utente(id) ON UPDATE CASCADE ON DELETE  
CASCADE;
```

22. Creazione tabella n_presenta

```
CREATE TABLE IF NOT EXISTS n_presenta (  
    seriale varchar(255) NOT NULL,  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL  
  
);  
  
ALTER TABLE n_presenta  
    ADD PRIMARY KEY (seriale, id_carta, id_rarita_carta),  
    ADD CONSTRAINT fk_n_presenta_1 FOREIGN KEY (seriale)  
REFERENCES busta(seriale) ON UPDATE CASCADE ON DELETE  
CASCADE,  
    ADD CONSTRAINT fk_n_presenta_2 FOREIGN KEY (id_carta,  
id_rarita_carta) REFERENCES carta(id, id_rarita) ON  
UPDATE CASCADE ON DELETE CASCADE;
```

23. Creazione tabella categoria

```
CREATE TABLE IF NOT EXISTS categoria (  
    id BIGSERIAL NOT NULL,  
    nome varchar(255) NOT NULL  
  
);  
  
ALTER TABLE categoria  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_categoria UNIQUE (nome);
```

24. Creazione tabella magie_trappole

```
CREATE TABLE IF NOT EXISTS magie_trappole (  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL,  
    id_categoria integer NOT NULL  
);  
  
ALTER TABLE magie_trappole  
    ADD PRIMARY KEY (id_carta, id_rarita_carta),  
    ADD CONSTRAINT fk_magie_trappole_1 FOREIGN KEY  
(id_carta, id_rarita_carta) REFERENCES carta(id,  
id_rarita) ON UPDATE CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_magie_trappole_2 FOREIGN KEY  
(id_categoria) REFERENCES categoria(id) ON UPDATE CASCADE  
ON DELETE CASCADE;
```

25. Creazione tabella attributo

```
CREATE TABLE IF NOT EXISTS attributo (  
    id BIGSERIAL NOT NULL,  
    nome varchar(255) NOT NULL  
);  
  
ALTER TABLE attributo  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_attributo UNIQUE (nome);
```

26. Creazione tabella sotto_tipo

```
CREATE TABLE IF NOT EXISTS sotto_tipo (  
    id BIGSERIAL NOT NULL,  
    nome varchar(255) NOT NULL  
);  
  
ALTER TABLE sotto_tipo  
    ADD PRIMARY KEY (id),  
    ADD CONSTRAINT unica_sotto_tipo UNIQUE (nome);
```

27. Creazione tabella mostro

```
CREATE TYPE mostri_livello AS ENUM ('mostro_normale',  
'mostro_effetto', 'mostro_fusione', 'mostro_synchro',  
'mostro_rituale');
```

```
CREATE TYPE mostri AS ENUM ('livello', 'xyz', 'link');
```

```
CREATE TABLE IF NOT EXISTS mostro (  
    id_carta integer NOT NULL,  
    id_rarita_carta integer NOT NULL,  
    tipologia_mostro mostri DEFAULT NULL,  
    tipologia_mostro_livello mostri_livello DEFAULT NULL,  
    link_level integer DEFAULT NULL,  
    rango integer DEFAULT NULL,  
    id_sotto_tipo integer NOT NULL,  
    id_attributo integer NOT NULL  
);
```

```
ALTER TABLE mostro  
    ADD PRIMARY KEY (id_carta, id_rarita_carta),  
    ADD CONSTRAINT fk_mostro_1 FOREIGN KEY (id_carta,  
id_rarita_carta) REFERENCES carta(id, id_rarita) ON  
UPDATE CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_mostro_2 FOREIGN KEY  
(id_sotto_tipo) REFERENCES sotto_tipo(id) ON UPDATE  
CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_mostro_3 FOREIGN KEY (id_attributo)  
REFERENCES attributo(id) ON UPDATE CASCADE ON DELETE  
CASCADE;
```

Inserimento dati

```
INSERT INTO rank(nome, range_min, range_max) VALUES
    ('rookie', 0, 199),
    ('bronze', 200, 699),
    ('silver', 700, 1199),
    ('gold', 1200, 1699),
    ('platinum', 1700, 9999);
```

```
INSERT INTO utente(id_rank, nome, nickname, gemme, elo)
VALUES
    (2, 'Federico', 'Fede', 0, 500),
    (1, 'Gabriele', 'Cuso', 156, 180),
    (4, 'Alessandro', 'Giacco', 862, 1456);
```

```
INSERT INTO amicizia(id_utente_1, id_utente_2) VALUES
    (1, 3),
    (1, 2);
```

```
INSERT INTO amichevole(id_utente_vincitore, data) VALUES
    (3, '2021-02-28'),
    (1, '2020-07-15');
```

```
INSERT INTO torneo(id) VALUES
    (1),
    (2),
    (3),
    (4);
```

```
INSERT INTO classificata(id_utente_vincitore, id_torneo,
data, premio) VALUES
    (3, 1, '2021-08-14', 500),
    (1, 2, '2020-12-15', 500),
    (1, 3, '2022-01-28', 1000),
    (3, 4, '2018-08-28', 1000);
```

```
INSERT INTO partecipa_classificata(id_utente,  
id_classificata) VALUES  
    (1, 1),  
    (2, 1),  
    (3, 1),  
    (1, 2),  
    (2, 2),  
    (3, 2);
```

```
INSERT INTO mazzo(id_utente, nome, n_carte) VALUES  
    (1, 'EROE Elementale', 55),  
    (1, 'Exodia', 40),  
    (2, 'Drago Bianco Occhi Blu', 40),  
    (2, 'NEOS', 40),  
    (3, 'Fantasmatrucco', 40),  
    (3, 'Drago Polvere di Stelle', 40);
```

```
INSERT INTO rarita(nome) VALUES  
    ('Ultra'),  
    ('Secret'),  
    ('Ultimate');
```

```
INSERT INTO carta(id_rarita, nome) VALUES  
    (1, 'Elemental HERO Neos'),  
    (1, 'Yubel'),  
    (1, 'Elemental HERO Spirit of Neos'),  
    (1, 'Elemental HERO Honest Neos'),  
    (1, 'Elemental HERO Neos Alius'),  
    (3, 'Elemental HERO Stratos'),  
    (3, 'Elemental HERO Liquid Soldier'),  
    (3, 'Elemental HERO Solid Soldier'),  
    (3, 'Elemental HERO Blazeman'),  
    (2, 'Elemental HERO Shadow Mist'),  
    (2, 'Neo Space Connector'),  
    (2, 'Wroughtweiller'),  
    (3, 'A Hero Lives'),  
    (3, 'Neos Fusion'),  
    (2, 'Fusion Recovery'),  
    (2, 'Polymerization'),  
    (2, 'Legacy of a HERO'),  
    (2, 'Miracle Fusion'),  
    (2, 'Fifth Hope'),  
    (2, 'Hero Signal'),
```

```
(2, 'Elemental Recharge'),  
(2, 'Magistry Alchemist'),  
(2, 'Elemental HERO Neos Kluger'),  
(2, 'Elemental HERO Great Tornado'),  
(2, 'Elemental HERO Nova Master'),  
(2, 'Elemental HERO The Shining'),  
(1, 'Elemental HERO Escuridao'),  
(1, 'Elemental HERO Absolute Zero'),  
(2, 'Elemental HERO Brave Neos'),  
(2, 'Elemental HERO Sunrise'),  
(1, 'Elemental HERO Neos Knight'),  
(2, 'Elemental HERO Gaia'),  
(2, 'Elemental HERO Grandmerge'),  
(1, 'Xtra HERO Dread Decimator'),  
(1, 'Elemental HERO Stratos'),  
(1, 'Elemental HERO Liquid Soldier'),  
(1, 'Elemental HERO Solid Soldier'),  
(1, 'Elemental HERO Blazeman'),  
(1, 'Fusion Recovery'),  
(1, 'Polymerization'),  
(1, 'Legacy of a HERO'),  
(1, 'Miracle Fusion'),  
(1, 'Fifth Hope'),  
(1, 'Hero Signal'),  
(3, 'Polymerization');
```



```
INSERT INTO carta_collezione(id_utente, id_carta,  
id_rarita_carta, qnt) VALUES  
  (3, 1, 1, 3),  
  (3, 2, 1, 1),  
  (3, 3, 1, 2),  
  (3, 4, 1, 2),  
  (3, 5, 1, 2),  
  (3, 6, 3, 2),  
  (3, 7, 3, 2),  
  (3, 8, 3, 2),  
  (3, 9, 3, 2),  
  (3, 10, 2, 2),  
  (3, 11, 2, 2),  
  (3, 12, 2, 1),  
  (3, 13, 3, 1),  
  (3, 14, 3, 3),  
  (3, 15, 2, 1),  
  (3, 16, 2, 3),  
  (3, 17, 2, 1),  
  (3, 18, 2, 1),  
  (3, 19, 2, 2),  
  (3, 20, 2, 2),  
  (3, 21, 2, 1),  
  (3, 22, 2, 2),  
  (3, 23, 2, 1),  
  (3, 24, 2, 1),  
  (3, 25, 2, 1),  
  (3, 26, 2, 1),  
  (3, 27, 1, 1),  
  (3, 28, 1, 1),  
  (3, 29, 2, 1),  
  (3, 30, 2, 1),  
  (3, 31, 1, 3),  
  (3, 32, 2, 1),  
  (3, 33, 2, 2),  
  (3, 34, 1, 1),  
  (3, 35, 1, 1),  
  (3, 36, 1, 1),  
  (1, 15, 2, 1),  
  (1, 45, 3, 2);
```

```
INSERT INTO mercato (id_utente, id_utente_carta_coll,
id_carta_carta_coll, id_rarita_carta_coll, qnt, prezzo)
VALUES
    (3, 3, 5, 1, 1, 50),
    (3, 3, 8, 3, 1, 50),
    (1, 1, 15, 2, 1, 75),
    (1, 1, 45, 3, 1, 75);
```

```
INSERT INTO apparizioni (id_mazzo, id_utente_carta_coll,
id_carta_carta_coll, id_rarita_carta_coll, n) VALUES
    (1, 3, 5, 1, 2),
    (1, 3, 8, 3, 3),
    (1, 3, 4, 1, 1),
    (1, 3, 3, 1, 1),
    (1, 3, 2, 1, 1),
    (1, 3, 1, 1, 1);
```

```
INSERT INTO espansione(nome, anno) VALUES
    ('Eroe Elementale', 2000),
    ('Tempesta Occhi Blu', 2005);
```

```
INSERT INTO compare(id_espansione, id_carta,
id_rarita_carta) VALUES
    (1, 1, 1),
    (1, 2, 1),
    (1, 3, 1),
    (1, 4, 1);
```

```
INSERT INTO busta(seriale, id_espansione, is_speciale)
VALUES
    ('CHACHA2000', 1, true),
    ('VARVAR2005', 2, false);
```

```
INSERT INTO comprare(data, id_utente) VALUES
    ('2019-05-18', 3),
    ('2019-12-25', 3);
```

```
INSERT INTO venire(seriale_busta, id_utente_comprare,
data_comprare) VALUES
    ('CHACHA2000', 3, '2019-05-18'),
    ('CHACHA2000', 3, '2019-12-25');
```

```
INSERT INTO s_presenta(seriale, id_carta,  
id_rarita_carta) VALUES  
    ('CHACHA2000', 1, 1),  
    ('CHACHA2000', 2, 1),  
    ('CHACHA2000', 3, 1),  
    ('CHACHA2000', 4, 1);
```

```
INSERT INTO acquistata(seriale, id_utente) VALUES  
    ('CHACHA2000', 3),  
    ('CHACHA2000', 1);
```

```
INSERT INTO categoria(nome) VALUES  
    ('normale'),  
    ('continua'),  
    ('terreno'),  
    ('rapida');
```

```
INSERT INTO magie_trappole(id_carta, id_rarita_carta,  
id_categoria) VALUES  
    (13, 3, 1),  
    (14, 3, 1),  
    (15, 2, 1);
```

```
INSERT INTO attributo(nome) VALUES  
    ('acqua'),  
    ('luce'),  
    ('fuoco'),  
    ('oscurità'),  
    ('vento'),  
    ('terra');
```

```
INSERT INTO sotto_tipo(nome) VALUES  
    ('normale'),  
    ('guerriero'),  
    ('bestia alata');
```

```
INSERT INTO mostro(id_carta, id_rarita_carta,  
tipologia_mostro, tipologia_mostro_livello,  
id_sotto_tipo, id_attributo) VALUES  
  (1, 1, 'livello', 'mostro_effetto', 1, 2),  
  (3, 1, 'livello', 'mostro_effetto', 2, 2),  
  (4, 1, 'livello', 'mostro_effetto', 2, 5),  
  (23, 2, 'xyz', 'mostro_normale', 1, 1),  
  (24, 2, 'xyz', 'mostro_normale', 2, 1),  
  (25, 2, 'xyz', 'mostro_fusione', 2, 3),  
  (26, 2, 'link', 'mostro_fusione', 1, 3),  
  (27, 1, 'link', 'mostro_fusione', 3, 4),  
  (28, 1, 'livello', 'mostro_normale', 2, 2),  
  (29, 2, 'livello', 'mostro_normale', 3, 3),  
  (5, 1, 'livello', 'mostro_effetto', 2, 2),  
  (8, 3, 'livello', 'mostro_effetto', 2, 4);
```

Sono state omesse alcune query di inserimento finali per il test sul funzionamento di alcuni trigger (alcune istruzioni sono presenti nel file allegato: trigger_test.sql)

Punto A extra, uso e implementazione dei trigger

1. Trigger sul limite delle carte in un mazzo (INSERT)

Gestisce il limite di carte all'interno del mazzo. Il minimo consentito è 40 mentre il massimo consentito è 60. Per evitare di portare a termine l'operazione nonostante ci sia un errore è stato utilizzato il costrutto BEFORE INSERT anziché AFTER

```
CREATE OR REPLACE FUNCTION limite_carte() RETURNS TRIGGER
AS $limite_carte$
    BEGIN
        IF (NEW.n_carte < 40) THEN
            RAISE EXCEPTION 'Minimum limit of cards in the
deck is 40';
        ELSEIF (NEW.n_carte > 60) THEN
            RAISE EXCEPTION 'Maximum limit of cards in the
deck is 60';
        END IF;
        RETURN NULL;
    END;
$limite_carte$ LANGUAGE plpgsql;

CREATE TRIGGER limite_carte
BEFORE INSERT ON mazzo
FOR EACH ROW EXECUTE FUNCTION limite_carte();
```

2. Trigger sul limite delle carte in un mazzo (UPDATE)

Limitazione delle carte in un mazzo ad un minimo di 40 e un massimo di 60 (before update).

```
CREATE OR REPLACE FUNCTION limite_carte_update() RETURNS
TRIGGER
AS $limite_carte_update$
    BEGIN
        IF (NEW.n_carte < 40) THEN
            RAISE EXCEPTION 'Minimum limit of cards in the
deck is 40';
        ELSEIF (NEW.n_carte > 60) THEN
            RAISE EXCEPTION 'Maximum limit of cards in the
deck is 60';
        END IF;
        RETURN NULL;
    END;
$limite_carte_update$ LANGUAGE plpgsql;

CREATE TRIGGER limite_carte_update
BEFORE UPDATE ON mazzo
FOR EACH ROW EXECUTE FUNCTION limite_carte_update();
```

3. Aggiornamento del rank dopo un cambiamento dell'elo

Questo trigger serve per accodare alla modifica dell'elo di un utente anche la modifica del relativo rank se necessario. Questo è stato fatto ipotizzando che al termine di una partita e quindi del relativo premio, positivo o negativo, per il vincitore e il perdente venga aggiornato subito dopo: l'elo di un utente. Quindi se esso supera il ranking attuale dell'utente allora il rank verrà aggiornato grazie a questo trigger (vale lo stesso anche per il declassamento).

```
CREATE OR REPLACE FUNCTION elo() RETURNS TRIGGER
AS $elo$
    BEGIN
        IF (OLD.elo != NEW.elo) THEN
            IF (NEW.elo < 200) THEN
                UPDATE utente SET id_rank=1 WHERE
nickname=NEW.nickname;
            ELSEIF (NEW.elo >= 200 AND NEW.elo < 700) THEN
                UPDATE utente SET id_rank=2 WHERE
nickname=NEW.nickname;
            ELSEIF (NEW.elo >= 700 AND NEW.elo < 1200) THEN
                UPDATE utente SET id_rank=3 WHERE
nickname=NEW.nickname;
            ELSEIF (NEW.elo >= 1200 AND NEW.elo < 1700) THEN
                UPDATE utente SET id_rank=4 WHERE
nickname=NEW.nickname;
            ELSEIF (NEW.elo >= 1700) THEN
                UPDATE utente SET id_rank=5 WHERE
nickname=NEW.nickname;
            END IF;
        END IF;
        RETURN NULL;
    END;
$elo$ LANGUAGE plpgsql;

CREATE TRIGGER elo
AFTER UPDATE ON utente
FOR EACH ROW EXECUTE FUNCTION elo();
```

4. Limite massimo di apparizioni di una carta in deck (INSERT)

Questo trigger limita il numero massimo di apparizioni delle carte all'interno di un deck (sono stati omessi nelle seguenti query di selezione i controlli sull'appartenenza allo stesso utente e allo stesso mazzo per una migliore formattazione e per brevità, ma era sufficiente inserire due condizioni in più nel WHERE). Il trigger in questione non permette di inserire carte con stesso nome per più di tre volte all'interno di uno stesso deck di uno stesso utente tenendo conto anche della rarità, ovvero se viene inserita una carta con stesso nome ma rarità differente, nel mazzo verrà contato che la somma delle apparizioni dei due siano minori o uguali a 3. È stato fatto tutto ciò grazie all'utilizzo del costrutto DECLARE alle cui "variabili" sono stati assegnati dei valori tramite query di selezione.

```
CREATE OR REPLACE FUNCTION limite_3_carte_mazzo() RETURNS
TRIGGER
AS $limite_3_carte_mazzo$
DECLARE boxid integer;
DECLARE altro_n integer;
BEGIN

    IF (NEW.n > 3) THEN
        RAISE EXCEPTION 'Maximum limit for the same card
reached';
    END IF;

    SELECT a.id_carta_carta_coll INTO boxid
    FROM apparizioni AS a, carta_collezione AS cc, carta
AS c, mazzo AS m
    WHERE m.id=a.id_mazzo AND
a.id_carta_carta_coll=cc.id_carta AND
a.id_rarita_carta_coll=cc.id_rarita_carta AND
a.id_utente_carta_coll=cc.id_utente AND c.id=cc.id_carta
AND cc.id_rarita_carta=c.id_rarita AND
c.nome=(SELECT c1.nome
        FROM carta_collezione AS cc1, carta AS c1
        WHERE c1.id=cc1.id_carta AND
        cc1.id_rarita_carta=c1.id_rarita AND
        cc1.id_carta=NEW.id_carta_carta_coll);

    SELECT a.n INTO altro_n FROM apparizioni AS a WHERE
a.id_carta_carta_coll=boxid;
```



```

IF( (NEW.n + altro_n) > 3) THEN
    RAISE EXCEPTION 'Maximum limit for the same card
    reached';
END IF;

RETURN NULL;
END;
$limite_3_carte_mazzo$ LANGUAGE plpgsql;

CREATE TRIGGER limite_3_carte_mazzo
AFTER INSERT ON apparizioni
FOR EACH ROW EXECUTE FUNCTION limite_3_carte_mazzo();

```

5. Limite massimo di apparizioni di una carta in deck (UPDATE)

Questo trigger è speculare a quello precedente (punto 4) solo che è stata aggiunta una condizione nel WHERE per il fatto che la carta che sta venendo inserita non dev'essere trovata dalla query di selezione. Questo problema sorge dal fatto che per un UPDATE il dato è già presente in tabella mentre nel caso precedente (INSERT) non è possibile trovare la carta che sta venendo inserita in quanto l'operazione dev'essere ancora portata a termine.

```

CREATE OR REPLACE FUNCTION limite_3_carte_mazzo_update()
RETURNS TRIGGER
AS $limite_3_carte_mazzo_update$
DECLARE boxid integer;
DECLARE altro_n integer;
BEGIN
    SELECT a.id_carta_carta_coll INTO boxid
    FROM apparizioni AS a, carta_collezione AS cc,
    carta AS c, mazzo AS m
    WHERE m.id=a.id_mazzo AND
    a.id_carta_carta_coll=cc.id_carta AND
    a.id_rarita_carta_coll=cc.id_rarita_carta AND
    a.id_utente_carta_coll=cc.id_utente AND
    c.id=cc.id_carta AND cc.id_rarita_carta=c.id_rarita
    AND a.id_carta_carta_coll <> NEW.id_carta_carta_coll
    AND c.nome=(SELECT c1.nome FROM carta_collezione AS
    cc1, carta AS c1 WHERE c1.id=cc1.id_carta AND
    cc1.id_rarita_carta=c1.id_rarita AND
    cc1.id_carta=NEW.id_carta_carta_coll);

    SELECT a.n INTO altro_n
    FROM apparizioni AS a
    WHERE a.id_carta_carta_coll=boxid;

```

```

        IF (NEW.n > 3) THEN
            RAISE EXCEPTION 'Maximum limit for the same
card reached';
        END IF;
        IF ((NEW.n + altro_n) > 3) THEN
            RAISE EXCEPTION 'Maximum limit for the same
card reached';
        END IF;
        RETURN NULL;
    END;
$limite_3_carte_mazzo_update$ LANGUAGE plpgsql;

CREATE TRIGGER limite_3_carte_mazzo_update
AFTER UPDATE ON apparizioni
FOR EACH ROW EXECUTE FUNCTION
limite_3_carte_mazzo_update();

```

6. Limite massimo di acquisti di una busta speciale

Uno stesso utente può comprare massimo 5 bustine speciali all'interno di una stessa settimana che va dal giorno attuale fino al lunedì precedente. Questa condizione è stata fatta tramite l'istruzione: `data_comprare BETWEEN date_trunc('week', current_date) AND CURRENT_DATE;` . Per il corretto funzionamento e la coerenza di questo trigger si ipotizza che a livello applicativo, se viene fatto un inserimento nella tabella “comprare” allora esso dev’essere fatto anche nella tabella “venire” subito dopo. Il tutto dev’essere gestito come un’operazione unica: ovvero se l’inserimento su “venire” non va a buon fine allora quello fatto su comprare verrà annullato. Per questo il trigger annullerà l’operazione di inserimento nella tabella “venire” ma non sulla tabella “comprare” perchè si ipotizza che l’azione venga fatta a livello applicativo.

```

CREATE OR REPLACE FUNCTION limite_5_buste() RETURNS
TRIGGER
AS $limite_5_buste$
DECLARE cronologia integer;
BEGIN

SELECT
    COUNT(*) INTO cronologia
FROM
    venire
WHERE

```

```
        id_utente_comprare=NEW.id_utente_comprare AND  
        seriale_busta=NEW.seriale_busta AND  
        data_comprare BETWEEN date_trunc('week',  
current_date) AND CURRENT_DATE;
```

```
IF (cronologia > 5) THEN  
    RAISE EXCEPTION 'Maximum limit of purchases  
reached';  
END IF;
```

```
    RETURN NULL;  
END;  
$limite_5_buste$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER limite_5_buste  
AFTER INSERT ON venire  
FOR EACH ROW EXECUTE FUNCTION limite_5_buste();
```